UNIVERSITÀ DEGLI STUDI DI MODENA

E REGGIO EMILIA

Dipartimento di Ingegneria "Enzo Ferrari"

Corso di Laurea Magistrale in Ingegneria Informatica (LM-32)

Integrazione di strategie di Continual Learning e Machine Unlearning per la mitigazione del concept drift in ML-NIDS

Relatore:
Prof. Mirco Marchetti
Correlatori:
Ing. Dimitri Galli
Prof. Mauro Andreolini

Candidato:
Luca Montagnani
Matricola 195431

ANNO ACCADEMICO 2024/2025

Sommario

I sistemi di Network Intrusion Detection basati su Machine Learning (ML-NIDS) soffrono di un rapido decadimento prestazionale dovuto alla natura non stazionaria del traffico di rete e al fenomeno del concept drift. In questo lavoro viene presentato un framework originale volto a garantire la resilienza e la scalabilità dei modelli ML-NIDS nel lungo periodo. L'approccio proposto integra metodologie di Continual Learning per l'acquisizione continua di nuove minacce senza incorrere nel catastrophic forgetting. Parallelamente, viene introdotta una tecnica di Machine Unlearning per la rimozione sistematica di dati obsoleti o inquinati, ottimizzando l'ingombro computazionale e preservando le prestazioni del sistema.

La cosa più importante non è il ricordo della tua esistenza, ma quello che hai fatto e ottenuto.

Indice

1 Introduzione **1**

1.1 Il problema del Concept Drift 1

1.2 Contributi 3

1.3 Organizzazione della tesi 4

2 Stato dell'Arte **5**

2.1 Attacchi informatici di rete: Botnet 5

2.2 Intrusion Detection System (IDS) 6

2.3 Machine Learning per NIDS 10

2.3.1 Pipeline di elaborazione 10

2.3.2 Algoritmi ML 11

2.4 Concept Drift 12

2.4.1 Concept Drift nella cybersecurity 18

2.4.2 Continual Learning 18

2.4.3 Machine Unlearning 23

2.5 Applicazioni nella sicurezza 27

3 Paradigma metodologico per la resilienza al drift tramite CL e MU **30**

3.1 Threat Model 31

3.1.1 Obiettivi del difensore 31

3.2 Integrazione di Continual Learning e Machine Unlearning 32

3.2.1 Obiettivi 32

3.2.2 Scalabilità 33

3.2.3 Resilienza del drift 34

3.2.4 Performance costanti 35

3.2.5 Gestione delle risorse 36

3.3 Formalizzazione dell'algoritmo 37

4 Realizzazione del framework: Integrazione dei processi di Replay e Pruning 38

4.1 Specifiche hardware e configurazione dell'ambiente di esecuzione 38

4.2 Dataset 39

4.2.1 Preprocessing 41

4.3 Classificatori 45

4.3.1 Configurazione dei classificatori 47

4.4 Implementazione 49

4.4.1 Training 50

4.4.2 Experience Replay e Data Pruning nell'ensemble 52

4.5 Scenari di Test 61

4.5.1 Pre-Deployment 62

4.5.2 Valutazione su traffico clean 62

4.5.3 Valutazione su traffico adversarial 63

5 Risultati 67

5.1 Descrizione delle metriche di valutazione 67

5.1.1 Metriche di valutazione 68

5.1.2 Area Under Time (AUT) 69

5.2 Pre-Deployment 70

5.3 Risultati su scenario clean 71

5.3.1 Concept Drift 71

5.3.2 Continual Learning 71

5.3.3 Machine Unlearning 72

5.3.4 Generalizzabilità del framework: Risultati su LR e MLP 76

5.3.5 Analisi dei risultati 76

5.4 Risultati su scenario adversarial 78

6 Conclusione 80

6.1 Discussione dei risultati 80

6.2 Sviluppi futuri 81

Ringraziamenti **83**

Riferimenti **84**

Elenco delle Figure

2.1 Classificazione degli IDS 7

2.2 Con l'aggiunta di piccole perturbazioni nei dati di input, è possibile aggirare il modello e indurlo ad una predizione sbagliata. 14

2.3 Esempio di concept drift 15

2.4 Sudden concept drift 16

2.5 Gradual concept drift 17

2.6 Incremental concept drift 17

2.7 Recurrent concept drift 17

2.8 Bilanciamento tra memory stability e learning plasticity al task k-esimo 20

2.9 Confronto tra processo di Retraining e Machine Unlearning 24

3.1 Rappresentazione schematica dell'approccio proposto: Integrazione tra CL e MU . 32

5.1 Performance della F1 nello scenario clean 73

5.2 Performance della Precision nello scenario clean 73

5.3 Performance della TNR (True Negative Rate) nello scenario clean 74

5.4 Performance della Recall nello scenario clean 74

5.5 Performance dell'AUT nello scenario clean 75

5.6 Confronto dell'evoluzione temporale del numero di modelli nell'ensemble. La linea dell'approccio proposto mostra l'effetto del Data Pruning nel mantenere la dimensionalità ridotta rispetto alla crescita lineare del solo Continual Learning. . . 77

Elenco dei Codici

4.1 Conversione di tutti i file `.pcap` in file `.argus` nella directory corrente 41

4.2 Conversione di tutti i file `.argus` in file `.csv` nella directory corrente 41

4.3 Inizializzazione del classificatore Random Forest 47

4.4 Inizializzazione del classificatore Logistic Regression con standardizzazione delle feature e pesatura delle classi. 47

4.5 Inizializzazione del modello Multi-Layer Perceptron con architettura a due strati nascosti e standardizzazione delle feature. 48

4.6 Downsampling della classe maggioritaria per ottenere un rapporto 1:1 51

4.7 Classificazione probabilistica dei dati di test 53

4.8 Ottenimento del valore massimo tra le probabilità ottenute con `np.max()` 54

4.9 Selezione dei primi K campioni più vicini al *decision boundary* 54

4.10 Utilizzo del buffer per compensare le mancanze di dati 57

4.11 Funzione per il calcolo dei pesi dei vari modelli dell'ensemble 57

4.12 Creazione di un nuovo modello addestrato sui dati nuovi 58

4.13 Analisi del contributo dei vari modelli dell'ensemble 60

4.14 Eventuale rimozione del modello in caso di superamento della soglia stabilita . . . 60

4.15 Funzione per l'avvelenamento delle feature 64

1. Introduzione

L'attuale panorama della sicurezza informatica è caratterizzato da una crescita esponenziale nella complessità e nella frequenza delle minacce dirette alle infrastrutture digitali. In questo scenario, i sistemi di Network Intrusion Detection (NIDS) rappresentano una linea di difesa fondamentale, agendo come sensori critici per l'identificazione di attività sospette all'interno dei flussi di dati [10]. In particolare, la protezione di reti aziendali di medie e grandi dimensioni richiede il monitoraggio costante di migliaia di host e l'analisi di volumi massicci di traffico.

Negli ultimi anni, l'integrazione di tecniche di Machine Learning (ML) ha permesso di superare i limiti strutturali dei sistemi tradizionali basati su firma (signature-based) [9], introducendo la capacità di rilevare minacce precedentemente ignote, come gli attacchi Zero-Day. Tuttavia, l'efficacia dei modelli ML-NIDS è strettamente legata a un'assunzione statistica fondamentale: la iid assumption (independent and identically distributed), secondo cui i dati di addestramento devono essere rappresentativi della distribuzione reale incontrata durante l'operatività [24].

1.1 Il problema del Concept Drift

La realtà operativa delle reti moderne viola sistematicamente questa assunzione a causa della natura non stazionaria del traffico. Il fenomeno del *Concept Drift*, ovvero il mutamento della relazione tra le caratteristiche dei dati di input e le etichette di classificazione — rappresenta la sfida principale per la sostenibilità dei sistemi di difesa nel lungo periodo [5]. L'evoluzione fisiologica dei protocolli di rete, l'adozione di nuovi servizi e il cambiamento delle tattiche adottate dagli attaccanti rendono i modelli statici rapidamente obsoleti.

Come dimostrato nelle analisi sperimentali di questa ricerca, un classificatore privo di meccanismi di aggiornamento subisce un decadimento drastico delle prestazioni dopo pochi mesi di attività, rendendo il sistema inaffidabile [4]. Tale obsolescenza non solo aumenta il rischio di falsi negativi, permettendo alle botnet di eludere il rilevamento, ma genera un volume ingestibile di falsi positivi, portando gli analisti del Security Operation Center (SOC) [14] verso una condizione di alert fatigue.

Affrontare il concept drift richiede un delicato bilanciamento tra la capacità di acquisire nuova conoscenza (plasticità) e la necessità di preservare quanto appreso in precedenza (stabilità) [22]. In un sistema di difesa ideale, il classificatore dovrebbe evolversi per riconoscere le nuove varianti di botnet senza però compromettere la sua efficacia sulle minacce storiche. Tuttavia, l'aggiornamento dei modelli di Machine Learning attraverso flussi sequenziali di dati introduce un ostacolo critico noto come *Catastrophic Forgetting* [21].

Il catastrophic forgetting si verifica quando l'apprendimento di nuove informazioni porta il modello a sovrascrivere o alterare pesantemente i pesi delle feature precedentemente ottimizzati per i task passati [21]. In termini di sicurezza di rete, questo significa che un NIDS, nel tentativo di adattarsi ai pattern di attacco di un determinato mese, potrebbe improvvisamente dimenticare come identificare botnet rilevate correttamente nei mesi precedenti.

Questo limite strutturale rende le strategie di retraining tradizionale spesso inefficaci a causa di una perdita di resilienza del modello, in quanto rimane vulnerabile a minacce ricorrenti o attacchi che riemergono a distanza di tempo; ma anche a causa dei costi computazionali, infatti addestrare costantemente un modello monolitico su un dataset che cresce indefinitamente, richiede risorse computazionali e di memoria insostenibili nel lungo periodo.

Per superare il problema del catastrophic forgetting, la tesi propone una soluzione basata sul Continual Learning, volta a garantire che l'integrazione di una nuova conoscenza avvenga in modo incrementale e protetto. Attraverso l'applicazione di una *Modular Network* e della tecnica di *Experience Replay* [22], il sistema preserva i campioni critici del passato anche nel lungo periodo, assicurando una difesa maggiormente stabile.

Per garantire la sostenibilità del framework nel lungo periodo, questa ricerca integra una metodologia di Machine Unlearning basata sulla tecnica del *Data Pruning* [25]. A differenza dei sistemi di difesa statici o puramente incrementali, l'unlearning non viene qui inteso solo come conformità normativa, ma come un processo di manutenzione proattiva della conoscenza. Attraverso un monitoraggio costante del contributo di ogni singolo modulo alle performance globali, il sistema è in grado di identificare ed eliminare fisicamente i componenti della rete modulare che risultano obsoleti, danneggiati o ridondanti.

L'applicazione del *Data Pruning* permette quindi di potare i rami della *Modular Network*, mantenendo costante il carico computazionale indipendentemente dalla durata del deployment. In

sintesi, il Machine Unlearning agisce come un filtro di qualità che assicura che il *decision boundary* dell'intero sistema sia definito solo dalla conoscenza più affidabile e aggiornata, trasformando il NIDS in un sistema capace non solo di imparare sequenzialmente, ma anche di dimenticare strategicamente ciò che non è più funzionale alla sicurezza della rete.

1.2 Contributi

Questa tesi si propone di fare luce su una problematica raramente indagata nel contesto dei ML-NIDS: l'interazione tra il decadimento temporale delle prestazioni (*concept drift*) e la resilienza a manipolazioni avversarie (*adversarial perturbations*). A tal fine, il lavoro presenta i seguenti contributi originali:

- È stato selezionato un dataset open-source documentato [20] (MCFP) che copre un arco temporale di 20 mesi (2016-2018), permettendo una valutazione realistica del concept drift che supera i limiti dei dataset sintetici a breve termine comunemente utilizzati in letteratura.
- È stata progettata un'architettura a Modular Network che integra l'Experience Replay per contrastare l'oblio catastrofico. Parallelamente, è stata introdotta una strategia di Data Pruning (ispirata ai principi SISA [6]) per la rimozione proattiva di moduli obsoleti o degradati.
- È stato simulato un attacco adversarial realistico di tipo "blind" manipolando feature grezze (Durata, Byte, Pacchetti) e garantendo la coerenza matematica dei flussi attraverso una pipeline di poisoning dedicata.
- Lo studio dimostra come la rimozione selettiva dei moduli non sia solo un'ottimizzazione computazionale, ma migliori anche le performance generali del sistema, fornendo una maggiore stabilità nella qualità delle classificazioni. Inoltre, si può notare un miglioramento anche nella capacità di rilevamento di campioni avversari rispetto ai classici approcci di solo apprendimento incrementale.
- L'efficacia del framework è stata verificata su diverse architetture (Random Forest, Logistic Regression e MLP), confermando la generalizzabilità della strategia proposta indipendentemente dal classificatore adottato .

Infine, le risorse e l'implementazione prodotte sono state rese pubbliche[1] per favorire la riproducibilità sperimentale e fornire una base per futuri sviluppi nel campo della difesa adattiva.

1.3 Organizzazione della tesi

Il presente lavoro è strutturati un sei capitoli, organizzati per condurre il lettore dall'analisi teorica del problema alla validazione sperimentale della soluzione proposta:

- **Capitolo 2: Stato dell'Arte**: Approfondisce il contesto dei sistemi di rilevamento delle intrusioni e le sfide poste dall'evoluzione dei dati nel tempo. Vengono esaminati il Machine Learning applicato ai NIDS, la tassonomia del *concept drift* e i paradigmi emergenti di *Continual Learning* e *Machine Unlearning*.

- **Capitolo 3: Metodologia**: Descrive l'architettura logica del framework proposto. Viene definito il *Threat Model* e illustrata l'integrazione sinergica tra *Modular Network*, *Experience Replay* e *Data Pruning* per garantire la resilienza del sistema.

- **Capitolo 4: Implementazione**: Illustra l'intero progetto sotto l'aspetto tecnico e realizzativo. Viene descritto il testbed hardware e software, insieme alla pipeline di preprocessing basata su Argus per l'estrazione di feature statistiche dai file PCAP del dataset MCFP. Il capitolo approfondisce le scelte ingegneristiche effettuate per la configurazione dei classificatori e la traduzione in codice delle strategie di *Experience Replay* e *Data Pruning*, fornendo una visione granulare di come il framework gestisce i flussi di dati e l'evoluzione della *Modular Network*.

- **Capitolo 5: Risultati**: Presenta l'analisi quantitativa delle prestazioni. Attraverso l'uso di diverse metriche temporali vengono confrontati gli scenari clean e adversarial, evidenziando il contributo del Machine Unlearning nel mantenimento della stabilità del sistema.

- **Capitolo 6: Conclusione**: Riassume i traguardi raggiunti, discutendo i limiti del lavoro e delineando le possibili direzioni per lo sviluppo futuro di sistemi di difesa perimetrale adattivi.

[1] `https://github.com/montaooo/Master-internship`

2. Stato dell'Arte

In questo capitolo viene discusso lo stato dell'arte relativo ai sistemi di rilevamento delle intrusioni e alle sfide poste dall'evoluzione dei dati nel tempo. Inizialmente, verranno presentati gli *Intrusion Detection Systems* (IDS), con particolare interesse sui sistemi che operano a livello di rete (NIDS). Successivamente, verrà analizzata l'integrazione del *Machine Learning* in tali sistemi, evidenziando come le assunzioni statistiche classiche si scontrino con i principi della cybersecurity. Infine, verrà approfondito il fenomeno del *Concept Drift*, esaminando le diverse tipologie esistenti e l'impatto che ha sulle performance dei modelli di difesa moderni.

2.1 Attacchi informatici di rete: Botnet

Prima di analizzare i sistemi di difesa, è necessario introdurre alla minaccia principale trattata in questa ricerca: Le *botnet*. Una botnet è una rete composta da dispositivi infettati da malware (chiamati bot o zombie), che sono controllati da un singolo attaccante [23].

A differenza dei malware tradizionali che colpiscono un singolo host, le botnet sono pensate per agire in modo coordinato su larga scala. La creazione di una botnet avviene in pochi passaggi:

1. Infezione: Ricerca di sistemi vulnerabili, utilizzo di phishing o attacchi brute-force.

2. Una volta infettato uno o più sistemi, viene installato il bot (malware) che contatterà periodicamente l'attaccante per ricevere istruzioni. Da notare che è il bot a comunicare, questo perché in questo modo può evitare più facilmente eventuali firewall o altri sistemi di sicurezza.

3. Una volta infettato il dispositivo, avviene l'esecuzione dell'attacco vero e proprio, che può variare da attacchi DDoS (*Distributed Denial of Service*) ad esfiltrazione di dati in massa, applicazione di ransomware, spam, ...

È chiaro quindi che è necessaria l'adozione di sistemi di monitoraggio e difesa perimetrale capaci di analizzare i flussi di dati in transito. È proprio in questo contesto che si inseriscono gli *Intrusion Detection Systems* (IDS), strumenti fondamentali per l'identificazione di malware sia negli host che nella rete.

2.2 Intrusion Detection System (IDS)

La difesa della rete e dei dispositivi da attacchi esterni o interni deve basarsi su operazioni di monitoraggio, organizzando tutta la rete interna per inviare dati, solitamente attraverso dei log. Dopo il monitoraggio, segue un processo per l'identificazione delle attività di intrusione, questa fase viene svolta dagli *Intrusion Detection Systems* (IDS).

Un IDS è un sistema che si occupa dell'analisi del traffico di rete ai vari livelli dello stack TCP/IP, dei log di sistema e di altri eventi (come l'apertura o il trasferimento di file), con l'obiettivo di individuare eventuali attività sospette. Di default, l'IDS è un sistema di monitoraggio passivo; infatti, esso fornisce una visibilità delle minacce ma non include attività di prevenzione o di risposta (a differenza degli *Intrusion Prevention Systems* o IPS)

In un sistema moderno di difesa, l'IDS è un sensore critico per tutta la catena di gestione degli incidenti:

- *Generazione di alert*: Tutti gli eventi che l'IDS identifica come potenziale attacco informatico generano degli allarmi di sicurezza.
- *SIEM (Security Information and Event Management)*: L'IDS invia tali allarmi al SIEM, un sistema di raccolta delle informazioni, progettato per raccogliere, aggregare e analizzare grandi volumi di dati in tempo reale [15].
- *SOC (Security Operation Center)*: È in continuo contatto con il SIEM. Si tratta di un team specializzato nella validazione degli allarmi e nella risposta alle minacce effettive [14].

Gli IDS possono essere classificati in base a diverse categorie [8] (Fig. 2.1). Nello specifico, si distinguono per:

- Obiettivo dell'analisi:
  - *Host Intrusion Detection System (HIDS)*: Monitorano le attività dei singoli dispositivi. Gli HIDS monitorano principalmente attività specifiche riguardanti i processi come system calls, log di sistema e file DLL (dynamic-link library) [18].

    Uno dei vantaggi principali degli HIDS è la capacità di analizzare i dati dopo che sono stati decifrati dall'host, permettendo di individuare minacce che un sistema di rete non

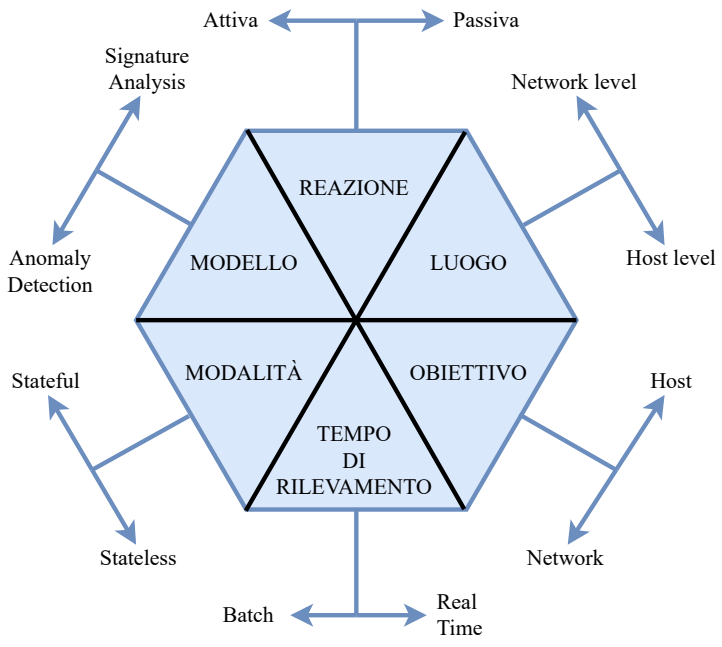

Figura 2.1: Classificazione degli IDS

riuscirebbe a vedere, perché nascoste all'interno di comunicazioni sicure (per esempio, HTTPS).

– *Network Intrusion Detection System (NIDS)*: I NIDS sono una delle prime linee di difesa, insieme ai firewall, in un tipico sistema di sicurezza. La loro funzione principale è quella di identificare le minacce in transito sui segmenti di rete prima che questi possano colpire gli host finali.

  L'efficacia dei NIDS, secondo lo studio di Garuba et al. [10], viene valutata in base alle capacità di garantire quattro pilastri fondamentali:

  1. Flessibilità e adattabilità: Un NIDS deve sapersi adattare all'evoluzione costante dei malware e delle vulnerabilità software.
  2. Rilevamento delle minacce esterne ed interne: A differenza dei firewall, i NIDS analizzano il traffico all'interno della rete, permettendo di individuare non solo le intrusioni dall'esterno, ma anche abusi di privilegi o movimenti laterali eseguiti da utenti interni.

3. Riduzione della superficie d'attacco: Attraverso l'analisi del traffico anomalo, i NIDS aiutano a identificare vulnerabilità nei protocolli o nei dispositivi.
4. Ispezione dei protocolli: I NIDS effettuano un'analisi non solo sulle intestazioni dei pacchetti, ma anche sul flusso dei dati di un protocollo, cercando di determinare l'intento di un'attività piuttosto che limitarsi al confronto di una stringa di byte.

- Luogo in cui avviene l'analisi:
  - *Host level*: Se agisce su un singolo host.
  - *Network level*: Se l'analisi controlla molteplici host nella rete.
- Tempo di rilevamento dei dati:
  - *Batch*: Gli eventi vengono accumulati in un posto prima di essere trasferiti per essere analizzati. Questo può avvenire in predeterminati intervalli di tempo o in predeterminate soglie che devono essere raggiunte [11].
  - *Real Time*: Come suggerisce il nome, l'analisi degli eventi avviene istantaneamente. Con questo approccio, non c'è delay nell'integrazione degli eventi, in quanto viene dato un risultato appena il dato viene ricevuto [11].
- Modalità di analisi:
  - *Stateless*: Ogni evento viene considerato indipendente dagli altri. Approccio veloce, leggero ed efficiente.
  - *Stateful*: Gli eventi vengono considerati a flussi, permettendo al sistema di individuare un contesto tra di essi. Questo approccio permette di individuare attacchi più complessi, come per esempio un attacco frammentato su pacchetti diversi.
- Tipi di reazione:
  - *Passiva*: Il sistema monitora il traffico senza interferire con il flusso dei dati. In caso di rilevamento, genera allarmi e log per gli analisti di sicurezza. È l'approccio più comune per evitare interruzioni di servizio dovute a falsi positivi

- *Attiva*: L'IDS intraprende azioni correttive automatiche, come l'interruzione delle sessioni TCP o la riconfigurazione dinamica delle regole del firewall. Quando un sistema è progettato per operare in-line e bloccare attivamente i pacchetti malevoli prima che raggiungano il target, viene comunemente definito come *Intrusion Prevention System (IPS)*.

- Tipo di modello adottato:
  - *Signature Analysis*: Vengono salvati i dati relativi alle operazioni effettuate da utenti autorizzati e da attaccanti informatici, creando quindi dei pattern (firme) di comportamenti normali e malevoli. È possibile identificare un utente non autorizzato da uno autorizzato attraverso il confronto di queste firme con gli eventi correnti [9].

    Il problema principale di questo modello è che se il pattern di accesso di un utente autorizzato cambia o se compare un nuovo attacco informatico non presente nel database dell'IDS (per esempio, un attacco *zero-day*), non si può determinare se l'utente è autorizzato o no.
  - *Anomaly Detection*: In questo caso, si classificano gli utenti in base al loro comportamento (può basarsi sul quantitativo di traffico generato, il contenuto di file condivisi o scaricati, e il tempo di utilizzo della rete). Vengono quindi segnalate come intrusioni tutte le deviazioni significative dal comportamento normale [9].

    Sebbene questi modelli permettano di identificare attaccanti che utilizzano attacchi *zero-day*, la loro efficacia è strettamente legata alla capacità del modello di distinguere un reale attacco da un mutamento fisiologico del traffico di rete, generando tendenzialmente più falsi positivi.

In questa tesi, ci concentreremo sui Network Intrusion Detection Systems (NIDS), una componente indispensabile per la protezione perimetrale. Tuttavia, un approccio tradizionale basato su signature analysis si dimostra oggi insufficiente per fronteggiare la dinamicità delle reti e degli attacchi informatici. Come evidenziato precedentemente, l'efficacia di un NIDS basato su anomaly detection è strettamente dipendente dalla stabilità del comportamento normale che viene definito.

A venirci in contro alla complessità nel trovare un comportamento normale, la ricerca si è orientata verso l'integrazione di tecniche di *Machine Learning*.

2.3 Machine Learning per NIDS

L'integrazione del Machine Learning nei NIDS nasce per la necessità di superare i limiti dei sistemi tradizionali basati su firma (*signature-based*), che richiedono aggiornamenti manuali costanti e non sono in grado di rilevare attacchi *Zero-Day*. Al contrario, i sistemi basati su ML possono impostare un comportamento normale della rete e identificare come intrusioni le deviazioni da tale profilo (*anomaly-based*).

2.3.1 Pipeline di elaborazione

Un ML-NIDS moderno opera attraverso una pipeline strutturata [7]:

1. *Flow Exporter* (raccolta dati): Il sistema cattura i dati a livello di pacchetti (PCAP) tramite interfacce fisice o librerie (come `libpcap`). Tuttavia, a causa delle velocità di trasmissione elevate, solitamente si preferisce lavorare su dati aggregati come i *NetFlow*. Il Flow Exporter si occupa della trasformazione di flussi di pacchetti in flussi statici caratterizzati da attributi comuni (come indirizzo IP sorgente/destinatario, porta sorgente/destinataria, protocollo utilizzato, ...).

2. *Feature Engineering* (estrazione delle feature): Dopo l'ottenimento dei dati, vengono identificati gli attributi (feature) necessari per la classificazione. I dati grezzi vengono trasformati in vettori numerici che includono caratteristiche di base (come durata della comunicazione, protocollo) e feature derivate. La selezione accurata delle feature è molto importante per migliorare l'efficienza degli algoritmi di predizione e per ridurre la dimensionalità del dataset.

3. *ML modeling*: Il modello viene addestrato e riceve successivamente i vettori di feature, applicando algoritmi (come Random Forest, SVM, LR, ...) Per classificare il traffico di rete.

4. *Output*: Infine, il sistema produce una classificazione finale dei dati ricevuti. In contesti operativi, questo output andrà a produrre alert che verranno gestiti dal SOC o procedure di mitigazione automatica.

Nell'ultimo decennio sono state proposte molte soluzioni basate sul Machine Learning (ML) per migliorare l'efficacia dei NIDS, documentato sia da pubblicazioni scientifiche che da brevetti. Possiamo dividere i modelli adottabili in due categorie, metodi ML basati su *unsupervised learning*, apprezzati in quanto ottenere le label dei dati di una rete risulta difficile, e su *supervised learning*, più costosi in termini di deployment, ma possono fornire ottime prestazioni [5].

2.3.2 Algoritmi ML

Esistono diversi paradigmi di apprendimento per migliorare i NIDS. In particolare, analizzeremo le principali tecniche esistenti:

Decision Tree (DT)

Utilizzano una serie di regole per classificare il traffico. Come suggerisce il nome, questi modelli hanno una struttura ad albero, composta da nodi, rami e foglie. Ogni nodo rappresenta una feature, i rami rappresentano la decisione che l'algoritmo deve prendere per quel nodo, mentre le foglie indicano le possibili classi di appartenenza della feature. Modelli evoluti come *Random Forest* (RF), rappresentano oggi uno standard per la loro precisione e robustezza [1].

K-Nearest Neighbor (KNN)

Questo algoritmo si basa sulla similarità geometrica. Un dato viene classificato in base alla vicinanza statistica ai sample già noti. Una grossa sfida di questo algoritmo è la selezione del parametro k, in quanto se troppo piccolo renderebbe il modello vulnerabile all'overfitting, mentre se troppo grande rischia di portare a classificazioni errate dei dati [1].

Support Vectore Machines (SVM)

Le SVM mirano a trovare l'iperpiano con la separazione ottimale in uno spazio delle feature n-dimensionale. Vengono utilizzate per risolvere problemi sia lineari che non lineari.

K-Means

A differenza dei metodi precedenti, il clustering (come il K-Means) non richiede dati etichettati. Raggruppa i dati in base a centroidi di similarità. Risulta particolarmente utile in contesti di identificazione di attacchi *Zero-Day*, in quanto identifica i gruppi di dati che si discostano drasticamente dai cluster di traffico normale.

Reti Neurali Artificiali (ANN)

Sono ispirate alla struttura biologica dei neuroni, le ANN sono molto utili per modellare relazioni non lineari in dataset di grandi dimensioni. Tuttavia, presentano criticità legate ai tempi di addestramento. Per cercare di abbattere questi costi computazionali, sono state introdotte delle varianti ottimizzate, come l'*Extreme Learning Machine* (ELM) [1].

Metodi di Ensemble

Il concetto di *Ensemble Learning* parte dal presupposto che ogni classificatore possieda specifici punti di forza e di debolezza su diverse tipologie di attacchi. Integrando più modelli "deboli", attraverso algoritmi di votazione o pesatura, è possibile ottenere un classificatore forte e resiliente.

2.4 Concept Drift

Tuttavia, anche i modelli di ML per NIDS hanno diversi problemi, infatti il Machine Learning segue la *iid assumption* (independent and identically distributed random variables), proprietà che va a compromettere l'applicazione del ML nella cybersecurity.

La iid assumption ci dice che le variabili devono essere:

- Indipendenti: La conoscenza di una variabile non deve fornire informazioni di un'altra e viceversa.
- Identicamente distribuite: Tutte le variabili provengono dalla stessa distribuzione di probabilità, non ci devono essere dei trend generali (ad esempio, aumentano con il passare del tempo).

Nel Machine Learning, questo principio si riflette nel fatto che i dati utilizzati in fase di training devono essere simili ai dati di testing (rappresentativi), altrimenti le performance del modello saranno diverse rispetto a quelle attese.

La iid assumption [24] non è solo una teoria statistica, ma una necessità pratica per semplificare i calcoli di ottimizzazione. Nei problemi di ottimizzazione, l'assunzione di variabili indipendenti e identicamente distribuite va a semplificare la likelihood function. Se assumiamo che ogni dato x_i sia indipendente rispetto agli altri, la probabilità di osservare l'intero dataset corrisponde al prodotto delle probabilità dei singoli eventi:

$$L(\theta) = P(x_1, x_2, ..., x_n|\theta) = \prod_{i=1}^{n} P(x_i|\theta)$$

Senza l'indipendenza bisognerebbe calcolare probabilità condizionate a catena, rendendo il calcolo esponenzialmente più difficile.

I computer sono molto più efficienti a calcolare diverse somme rispetto a delle moltiplicazioni. Grazie alla proprietà dei logaritmi, è possibile trasformare il prodotto in una somma:

$$\log(L(\theta)) = \log(\prod P(x_i|\theta)) = \sum \log(P(x_i|\theta))$$

Inoltre, questa semplificazione migliora l'efficienza computazionale convertendo funzioni esponenziali in lineari (molte distribuzioni di probabilità contengono infatti termini esponenziali, che vengono semplificati dal logaritmo).

Infine, per massimizzare la probabilità dell'evento osservato, la funzione logaritmica viene applicata per massimizzare il parametro θ:

$$argmax_\theta \log(L(\theta))$$

Questo principio va a compromettere l'applicazione del ML in cybersecurity, perché interferisce con tre caratteristiche di questo dominio: *Scenari adversarial*, la *confidenzialità dei dati* e il *concept drift* [5].

- *Scenari adversarial*: La cybersecurity assume implicitamente la presenza di attaccanti. Nonostante la maggior parte degli attacchi siano "stazionari", esistono anche attaccanti che si

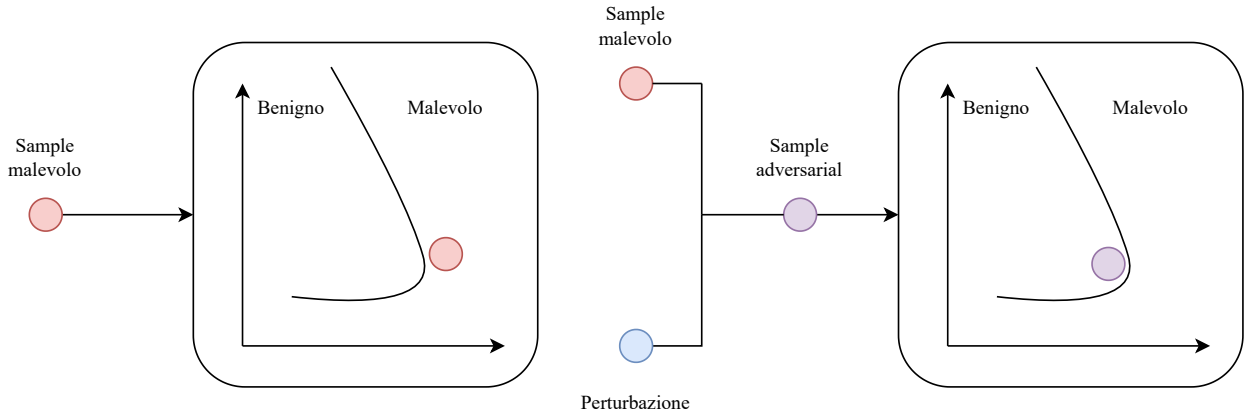

Figura 2.2: Con l'aggiunta di piccole perturbazioni nei dati di input, è possibile aggirare il modello e indurlo ad una predizione sbagliata.

aggiornano e cambiano le loro strategie di attacco. Inoltre, l'introduzione del ML nella cybersecurity ha esposto anche a minacce di *sample adversarial*, che hanno come obiettivo specificatamente i modelli di ML.

- *Confidenzialità*: Uno dei pilastri principali della cybersecurity è la confidenzialità dei dati. In relazione al ML, diventa un vincolo significativo, che possiamo dividere in due parti. Innanzitutto, l'utilizzo di crittografia rende l'analisi dei pacchetti più complessa, rendendo i modelli ML che si basano sull'analisi dei contenuti inefficaci. In secondo luogo, l'analisi dei pacchetti deve essere conforme alle normative sulla privacy (come GDPR e NIS2). L'obbligo di anonimizzare o rimuovere dati sensibili su richiesta, non solo complica la raccolta dei sample, ma rende difficile mantenere dataset rappresentativi e coerenti nel lungo periodo (compromettendo la iid assumption).

- *Concept drift*: Il concept drift è un'evoluzione dei dati che invalida la iid assumption. I sistemi moderni sono in costante evoluzione, così come la comunicazione tra essi, che può mutare a causa dell'introduzione di nuovi protocolli, aggiornamenti software, nuovi dispositivi e utenti. Ciò implica che la distribuzione dei dati osservata in fase di training differisce da quella reale incontrata in fase di testing, rendendo i modelli obsoleti o meno precisi.

In conclusione, emerge chiaramente come la iid assumption rappresenti un limite strutturale per sistemi di difesa basati su modelli statici. Sebbene gli sccenari adversarial e la confidenzialità dei dati pongono limiti immediati, è il concept drift a minare la validità del modello nel lungo periodo.

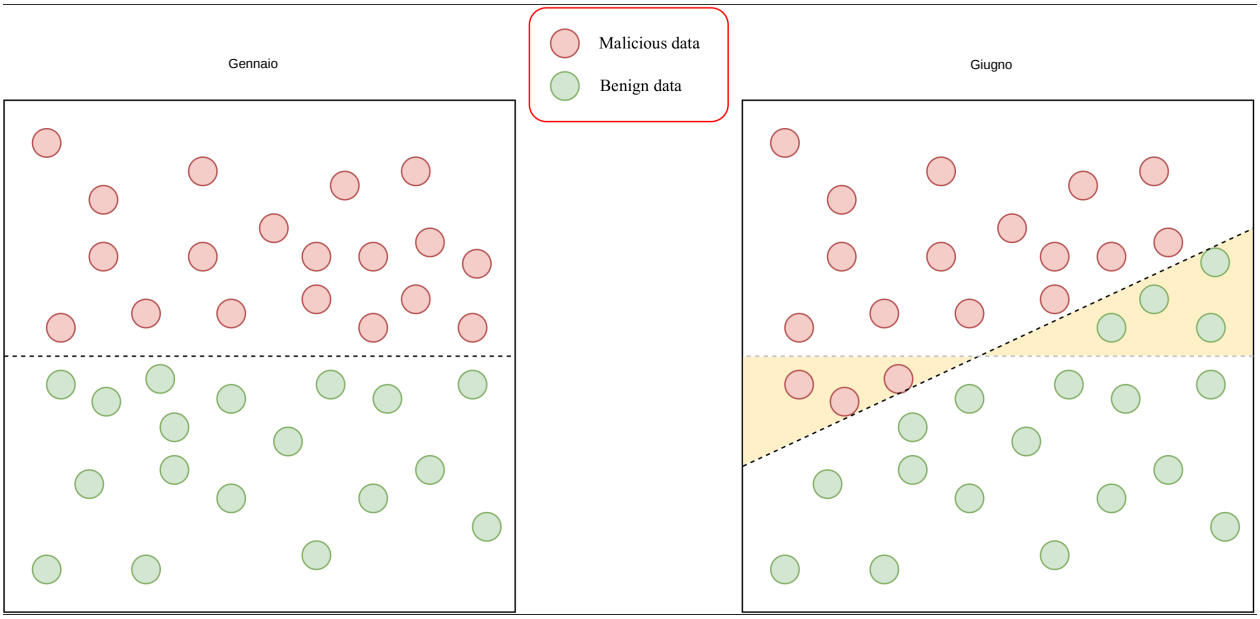

Figura 2.3: Esempio di concept drift

Nel prossimo capitolo verrà analizzato nel dettaglio questo fenomeno, esaminando le soluzioni proposte dalla letteratura scientifica, con particolare riferimento ai contributi di Apruzzese et al. [4].

Il concept drift consiste quindi in un cambiamento della relazione tra i dati di input e il target del modello.

Il diagramma in figura 2.3 mostra l'impatto del concept drift sullo lo spazio delle feature, dove il *decision boundary*, rappresentato dalla linea tratteggiata, rappresenta il confine appreso dal modello per distinguere il traffico benigno da quello malevolo. All'istante iniziale, i dati benigni e i dati malevoli sono correttamente separati, in quanto il modello è addestrato su dati recenti. Dopo sei mesi di operatività senza aggiornamenti, la distribuzione delle classi ha subito un mutamento, portando a cambiare anche l'effettivo decision boundary. Questo fenomeno evidenzia come un modello allenato su Gennaio inizi a produrre falsi positivi o falsi negativi in numero crescente a causa del concept drift.

Sebbene all'atto pratico le distinzioni semantiche non sono così importanti, è utile comprendere le diverse tipologie di cambiamenti (drift) che possono verificarsi. Nella pratica, infatti, questi fenomeni tendono a sovrapporsi, portando i professionisti del settore a utilizzare termini come *data drift* e *concept drift* in modo spesso intercambiabile [2].

- *Model drift*: Indica il degrado delle performance del modello, senza un motivo specifico. Il model drift è spesso causato dal concept drift.

- *Data drift*: Consiste nel cambio della distribuzione dei dati di input $P(X)$, a differenza del concept drift che consiste nel cambio delle relazioni tra gli input del modello e gli output $P(Y|X)$.

- *Target drift*: Spesso è sinonimo di concept drift. Può anche riferirsi al cambiamento della distribuzione delle label $P(Y)$.

Il concept drift può anche essere suddiviso per pattern, vediamo le tipologie esistenti:

- *Sudden concept drift*: Si verifica quando la distribuzione dei dati cambia in modo improvviso da un istante all'altro. Come si può notare in Figura 2.4, il sistema si trova a dover gestire una realtà completamente nuova senza alcun preavviso, rendendo i modelli precedenti immediatamente obsoleti.

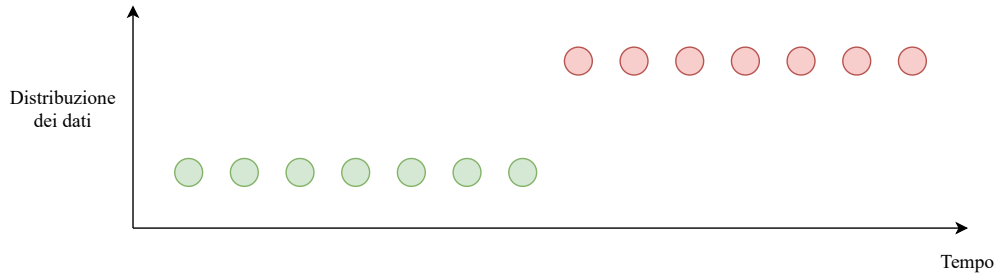

Figura 2.4: Sudden concept drift

- *Gradual concept drift*: Tipologia di drift più frequente, indica una transizione fluida tra eventi originali e eventi nuovi. Con il passare del tempo, la frequenza delle osservazioni legate agli eventi vecchi diminuisce progressivamente a favore di quelli nuovi (Figura 2.5).

- *Incremental concept drift*: A differenza del drift graduale, quello incrementale consiste in una mutazione continua degli eventi verso un nuovo stato. Come si può notare in Figura 2.6, la variazione avviene per piccoli passi, rendendo difficile stabilire un punto esatto di rottura ma portando comunque ad una divergenza significativa nel lungo periodo.

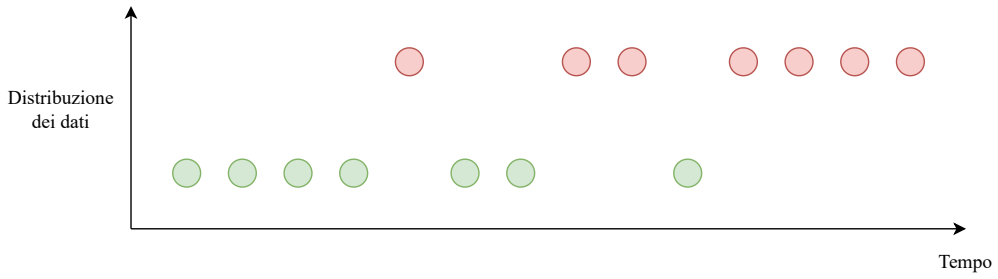

Figura 2.5: Gradual concept drift

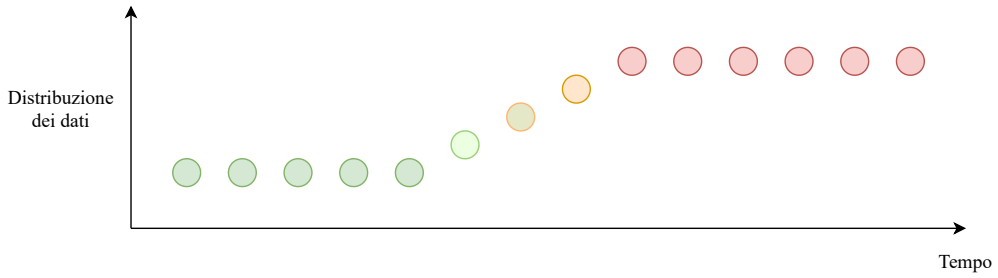

Figura 2.6: Incremental concept drift

- *Recurrent concept drift*: Pattern che si osserva quando determinati eventi si presentano con ciclicità (Figura 2.7). Questo scenario è tipico di attacchi stagionali o comportamenti di rete periodici, suggerendo di mantenere una "memoria" dei modelli passati per un possibile riutilizzo.

In questa tesi, pur riconoscendo la varietà delle tassonomie adottabili, si adotterà il termine *concept drift* come etichetta generale per descrivere il degrado delle performance dovuto alla natura non statica dei dati.

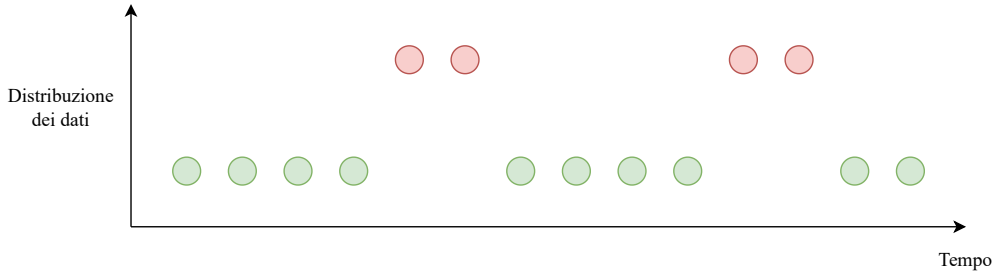

Figura 2.7: Recurrent concept drift

2.4.1 Concept Drift nella cybersecurity

Il Machine Learning applicato alla sicurezza informatica opera in un ambiente radicalmente diverso rispetto ad altri ambiti (come il riconoscimento di immagini o il filtraggio spam standard). Qui, la *iid assumption*, secondo cui i dati di test devono seguire la stessa distribuzione dei dati di training, viene sistematicamente violata.

Le cause principali per cui i dati di rete e i pattern dei malware presenti cambiano nel tempo possono essere riassunte in:

- Evoluzione fisiologica della rete: Le infrastrutture di rete non sono statiche, l'introduzione di nuovi protocolli, l'adozione di servizi cloud, il lavoro da remoto e l'aggiornamento dei sistemi operativi modificano il profilo del traffico "normale" che il modello deve essere in grado di riconoscere come benigno.

- Evoluzione delle minacce: Gli attaccanti, come le infrastrutture, si aggiornano per superare le nuove misure di sicurezza presenti.

- Stagionalità: Possono anche verificarsi dei picchi di traffico dovuti ad eventi specifici oppure dei backup pianificati, che possono alterare temporaneamente le feature statistiche di rete (come il carico della CPU o il volume di byte trasmessi), inducendo il modello a sbagliare.

Il concept drift nei modelli ML-NIDS è noto avere implicazioni importanti per la sicurezza, come un aumento dei falsi negativi, dovuto al fatto che gli attacchi "scivolano" sotto il *decision boundary* del modello in quanto presentano caratteristiche diverse rispetto a quelle apprese mesi prima; ma anche come un aumento dei falsi positivi, infatti se il traffico benigno cambia nel tempo e il modello lo scambia per malevolo, gli analisti SOC verranno poi sommersi da falsi allarmi, portando ad una maggiore correzione degli errori e ad una perdita di fiducia nel sistema (*Alert Fatigue*).

2.4.2 Continual Learning

Il *Continual Learning* è un paradigma del Machine Learning che permette ad un modello di apprendere nuovi dati sequenzialmente senza dimenticare le informazioni apprese precedentemente.

A differenza dei metodi tradizionali di apprendimento, il Continual Learning, come l'essere umano, si adatta a nuovi ambienti e esperienze. Tuttavia, questa strategia pone un problema principale: il *Catastrophic Forgetting*, un fenomeno che nasce con l'apprendimento di nuovi dati, perché tale apprendimento può portare il modello a peggiorare le sue prestazioni sui dati più vecchi a causa dell'aggiornamento dei pesi delle feature [21].

In generale, le performance del continual learning possono essere valutate attraverso tre aspetti: Performance generali, memory stability e learning plasticity [22].

- *Performance generali*: Tipicamente calcolate attraverso l'accuratezza e le sue performance derivate, precisione, recall e F1.

- *Memory stability*: Indica l'abilità del modello nel predire correttamente task passati nonostante l'apprendimento di nuovi. Può essere calcolata in base a *forgetting measure* (FM) e *backward trasnfer* (BWT).

  - *Forgetting measure*: Il *forgetting* di un task viene calcolato come la differenza massima tra le performance ottenute nel passato e le performance correnti sullo stesso task j:

    $$f_{j,k} = \max_{i \in 1,\dots,k-1} (p_{i,j} - p_{k,j}), \forall j < k$$

    La FM al k-esimo task è la media del forgetting dei task vecchi:

    $$FM_k = \frac{1}{k-1} \sum_{j=1}^{k-1} f_{j,k}$$

  - *Backward transfer*: Valuta l'influenza del learning al k-esimo task sui dati vecchi:

    $$BWT_k = \frac{1}{k-1} \sum_{j=1}^{k-1} (p_{k,j} - p_{j,j})$$

    In sostanza, la backward transfer non è altro che un forgetting negativo.

- *Learning plasticity*: Consiste nell'abilità del modello di evolversi e integrare nuove informazioni, senza essere "frenato" dai dati passati. Si calcola tramite la *intransience measure* (IM) e la *forward transfer* (FWT).

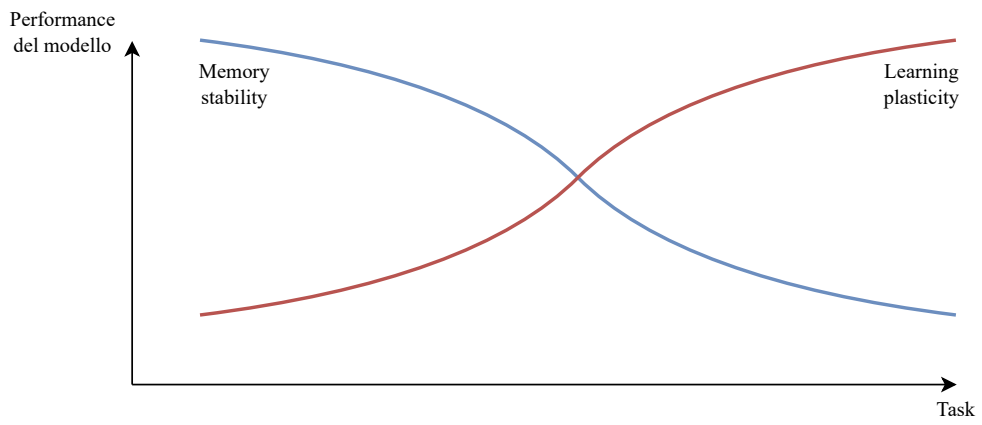

Figura 2.8: Bilanciamento tra memory stability e learning plasticity al task k-esimo

- *Intransience measure*: Inabilità del modello nell'apprendere nuovi task. Confronta le performance del modello addestrato solo sul task corrente rispetto a quelle del modello di Continual Learning.

$$IM_k = p_k - p_{k,k}$$

- *Forward transfer*: Valuta l'influenza dei task vecchi al task k-esimo:

$$FWT_k = \frac{1}{k-1} \sum_{j=2}^{k} (p_{j,j} - p_j)$$

Una delle sfide centrali del Continual Learning consiste proprio nel bilanciare questi due fattori, attraverso il cosiddetto *stability-plasticity trade-off*, in cui un'eccessiva learning plasticity o memory stability possono compromettere l'altra (Figura 2.8).

Ci sono diversi metodi applicabili nel Continual Learning per risolvere al problema del concept drift [22]:

Replay-based

Memorizzare o approssimare una piccola parte dei dati passati, per riutilizzarli durante l'addestramento dei nuovi dati. Questo approccio può essere suddiviso in tre ulteriori metodi:

- *Experience Replay*: Vengono salvati i sample dei task passati solitamente in un buffer di piccole dimensioni. Quindi, durante il training del task k, il modello riceve sia i nuovi dati

che il buffer contenente i dati vecchi.

- *Generative Replay*: Invece di salvare i dati reali, viene utilizzato un modello generativo per generare dei sample sintetici che simulano quelli passati. Utile per evitare problemi di privacy, in quanto non vengono memorizzati i dati reali. Tuttavia, dato che il continual learning di modelli generativi è molto difficile e può comportare overhead, il Generative Replay è tipicamente limitato a dataset relativamente semplici.

- *Feature Replay*: Evoluzione del Generative Replay, ottimizzata per ridurre la complessità computazionale. Il modello generativo viene spostato dal data-level al feature-level. In sostanza, il modello non vedrà nuovamente i dati originali, ma le sue caratteristiche già processate. In questo caso, il problema principale diventa il cosiddetto *representation shift*, in quanto le feature estratte in un determinato periodo potrebbero non essere più coerenti con l'attuale modo in cui il modello "vede" i dati [22].

Regularization-based

Questi metodi applicano dei termini di regolarizzazione per bilanciare i task vecchi con quelli nuovi:

- *Weight Regularization*: Consiste nel vincolare l'aggiornamento dei parametri (pesi) del modello identificati come cruciali per i task passati. Attraverso l'introduzione di una penalità nella funzione di perdita, si impedisce che l'apprendimento di nuove informazioni sovrascriva la conoscenza pregressa, mitigando così l'oblio catastrofico. Un esempio reale può essere EWC (Elastic Weight Consolidation).

- *Function Regularization*: Non si concentra sui pesi, ma sulla funzione di risposta (le predizioni). Ha quindi l'obiettivo di tenere l'output coerente con quanto appreso in passato. Un esempio in questo caso è LwF (Learning without Forgetting).

Optimization-based

Si basano sulla manipolazione dei programmi di ottimizzazione, come *gradient projection*, *loss landscape* e *meta-learning*.

Representation-based

Si basa sulla creazione di rappresentazione dei dati che siano robuste e ben generalizzate. L'idea principale è quella di rappresentare le feature in uno spazio in cui le nuove conoscenze apprese non vadano a sovrapporsi con quelle precedenti.

- *Self-Supervised Learning*: Sfrutta un apprendimento auto-supervisionato per ottenere rappresentazioni intrinsecamente più robuste al forgetting rispetto a quelle ottenute con apprendimento supervisionato standard.
- *Pre-training for Downstream Tasks*: Sfrutta modelli pre-addestrati su larga scala.
- *Continual Pre-Training*: Dato che il grande quantitativo di dati richiesti per il pre-training viene solitamente raccolto in modo incrementale, questa strategia mira ad eseguire un apprendimento continuo "a monte" per migliorare le prestazioni sui task che il modello dovrà affrontare in futuro.

Architecture-based

Questo approccio mira ad adattare la struttura del modello per i nuovi task, dedicando parti specifiche a specifiche attività. Possiamo dividerlo ulteriormente in tre sotto-categorie:

- *Parameter Allocation*: Si isolano sotto-parti (neuroni o rami del modello) specifiche per compiti specifici.
- *Model Decomposition*: Scompone il modello in due componenti, una parte condivisa e una specifica. La parte condivisa impara le caratteristiche comuni, mentre quella specifica è composta da componenti che devono imparare caratteristiche più specifiche per l'appunto (questa parte è tipicamente espandibile).
- *Modular Network*: Invece di utilizzare un unico modello, il sistema è composto da diversi sotto-moduli separati che lavorano in parallelo. Questo metodo permette di abbattere totalmente l'interferenza tra i moduli, a discapito della scalabilità a causa dell'espansione del modello, che richiede manutenzione.

Come si può notare, la scalabilità è una sfida trasversale per tutti i metodi di Continual Learning, particolarmente critica per approcci architecture-based, dove l'espansione del modello può portare ad un consumo insostenibile di risorse. Una soluzione a questo problema può essere il *Machine Unlearning*, che permette di bilanciare la crescita del modello con la rimozione di parti obsolete.

2.4.3 Machine Unlearning

Il Machine Unlearning è un paradigma che risponde alla necessità di rimuovere in modo definitivo (e selettivo) specifici dati e la loro influenza da un modello già addestrato su di essi. Mentre la rimozione di semplici dati da un database è un'operazione molto semplice, per quanto riguarda un modello di Machine Learning è tutt'altra cosa. Infatti, durante la fase di training, le informazioni vengono assimilate e distribuite tra svariati parametri interni al modello, rendendo il dato originale una parte integrante del sistema.

La necessità del Machine Unlearning deriva principalmente da tre fattori principali:

- *Conformità alle normative*: Normative recenti come il GDPR nell'Unione Europea hanno stabilito il *Diritto all'oblio*. Se un utente revoca il consenso al trattamento dei propri dati, l'organizzazione in questione non solo deve cancellare tali dati dai propri server, ma deve anche garantire che non influenzino più il comportamento dei propri modelli predittivi.

- *Sicurezza e Privacy*: Esistono diversi tipi di attacchi informatici volti a violare la riservatezza dei dati nei modelli di ML. Un primo esempio sono i *Membership Inference Attacks*, ossia attacchi che permettono di verificare con alta probabilità se un particolare dato è stato utilizzato o meno per addestrare un modello. Altrimenti esistono i *Model Inversion Attacks*, cioè attacchi che mirano a ricostruire parzialmente le feature dei dati di training interrogando il modello

- *Integrità e Manutenzione*: Infine, il Machine Unlearning risulta molto utile per la resilienza del modello. Infatti, se quest'ultimo dovesse essere stato vittima di un attacco di *Data Poisoning* o se dovesse aver appreso dati obsoleti, il Machine Unlearning permette di rimuovere tali dati e la loro influenza, riducendo le risorse computazionali utilizzate e migliorando le prestazioni del modello.

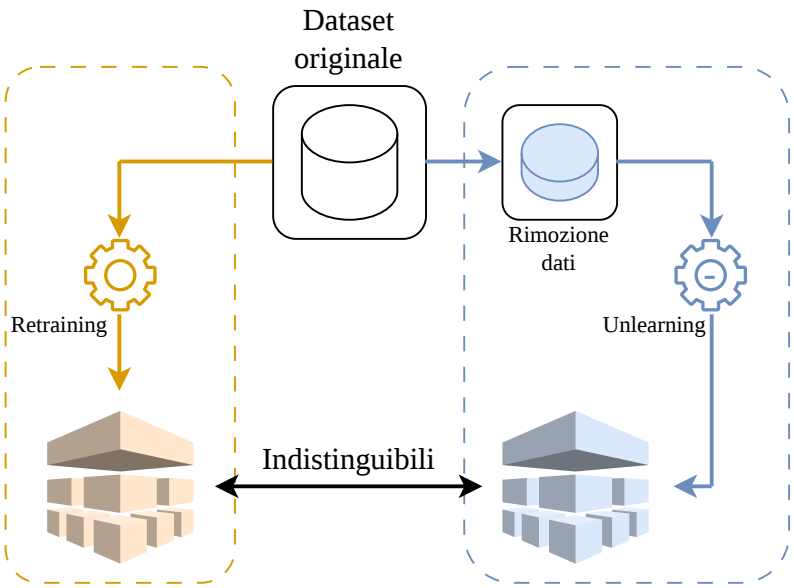

Figura 2.9: Confronto tra processo di Retraining e Machine Unlearning

L'obiettivo del Machine Unlearning è quindi quello di ottenere un modello che sia indistinguibile da uno addestrato da zero senza la rimozione dei dati, ma con un costo computazionale inferiore (Figura 2.9).

Il Machine Unlearning può essere quindi suddiviso in due macro-categorie: metodi di unlearning e meccanismi di verifica [25]. I metodi di unlearning si possono basare sulla modifica diretta dei dati o sulla manipolazione dei pesi del modello.

Data Reorganization

Si riferisce a tutte le tecniche che vanno ad agire sul dataset di training. Include tre metodi principali:

- *Data Obfuscation*: Mira a rendere i dati irriconoscibili attraverso l'introduzione di rumore. Invece che rimuovere fisicamente i dati quindi, questa strategia modifica i dati in modo che il modello non sia più in grado di estrarre informazioni utili da essi. Si può applicare a praticamente tutti i tipi di modelli, ma diventa complesso far dimenticare completamente le informazioni ai modelli.

- *Data Pruning*: Suddivide il training dataset in diversi sotto-dataset e allena corrispondentemente diversi sotto-modelli, ognuno per sotto-dataset. Ad una richiesta di unlearning, il modello rimuove i sample dai sotto-dataset che lo contengono per poi riaddestrare i sotto-

modelli affetti dalla modifica. Questo approccio di unlearning è molto semplice, veloce da applicare e garantisce intrinsecamente la rimozione completa di un dato, tuttavia richiede spazio di archiviazione aggiuntivo per conservare i vari sotto-modelli e i frammenti di dataset, inoltre l'accuratezza del modello potrebbe diminuire all'aumentare dei sotto-dataset, in quanto frammenti troppo piccoli rispetto al totale potrebbero non avere abbastanza informazioni per addestrare modelli parziali efficaci.

Un esempio di questo approccio è l'algoritmo SISA (Sharded, Isolated, Sliced and Aggregated training) [6]. L'obiettivo principale di questo algoritmo è di consentire di rimuovere completamente l'influenza di specifici dati da un modello di Machine Learning senza riaddestrare l'intero sistema da zero, garantendo che il modello finale sia prestazionalmente identico ad uno mai addestrato su quei dati.

- *Sharding*: Il dataset originale viene partizionato in S blocchi separati, chiamati *shards*. In particolare, ogni dato è contenuto in uno e un solo shard. Inoltre, invece che un unico modello centrale, vengono addestrati molteplici modelli, uno per shard.
- *Isolation*: Durante l'addestramento, i modelli operano in totale isolamento l'uno dall'altro. Non avviene alcuno scambio di informazioni tra i modelli dei diversi shard. In questo modo, si garantisce che l'influenza di un dato sia limitata esclusivamente ad un singolo modello, rendendo l'unlearning localizzato e facilmente verificabile.
- *Slicing*: Per ridurre ulteriormente i tempi di retraining, ogni shard viene ulteriormente suddiviso in R *slice* o fette. Il modello dello shard viene quindi addestrato incrementalmente sulle slice appartenenti a tale shard, salvando lo stato dei parametri (checkpoint) prima di introdurre un nuovo slice. In questo modo, se viene eliminato un dato in uno slice r, il modello non viene riaddestrato da zero, ma dall'ultimo checkpoint $(r - 1)$.
- *Aggregation*: Ora, il sistema è composto da tanti modelli indipendenti addestrati su shard di dati diversi. Ogni modello viene quindi testato su un nuovo set di dati, dando una propria previsione. Infine, i risultati vengono poi combinati tramite una votazione a maggioranza o tramite vettori di confidenza.

Poiché ogni shard ha una dimensione pari a $\frac{1}{S}$, il tempo del calcolo di retraining è ridotto a livello teorico di un fattore S. Infatti, non essendoci scambio di informazioni tra i diversi

modelli, se uno dovesse essere modificato, gli altri $S - 1$ modelli rimangono validi e non devono essere toccati.

- *Data Replacement*: Il dataset viene sostituito da un dataset trasformato. Permette anche questo approccio di rimuovere semplicemente tutte le informazioni da rimuovere. È però difficile rappresentare perfettamente il dataset originale attraverso la sostituzione delle informazioni.

Model Manipulation

Cerca di realizzare le operazioni di unlearning modificando i parametri del modello. Anche in questo caso, possiamo suddividerlo in tre categorie:

- *Model shifting*: L'idea principale è quella di calcolare l'influenza dei sample sui parametri del modello e dopodiché aggiornare i parametri per rimuovere tale influenza. In questo caso, se w sono i parametri del modello originale, $w_u = w + \delta$ corrisponde ai nuovi parametri, dove δ è il valore aggiornato. Non richiede molti parametri intermedi da mantenere, ma non è semplice trovare un offset appropriato per modelli complessi.

- *Model Replacement*: Il modello rimpiazza alcuni parametri con dei parametri pre-calcolati. Questi metodi dipendono solitamente da strutture di modelli specifici in modo da calcolare i parametri a priori. Il processo di unlearning viene completato velocemente, applicabile però solo ad un sottoinsieme di modelli a causa della necessità di calcolare in anticipo i parametri.

- *Model Pruning*: Vengono rimossi alcuni parametri dai modelli per dimenticare i sample necessari. Anche in questo caso, questo approccio è applicabile solo a strutture di modelli specifiche, in compenso è estremamente semplice rimuovere le informazioni da dimenticare.

I meccanismi di verifica sono estremamente importanti, in quanto i provider dei modelli devono poter garantire che il modello abbia dimenticato completamente i dati richiesti per la rimozione.

Valutazioni empiriche

- *Retraining-based*: Il retraining può provvedere in modo intrinseco alla verificabilità dell'unlearning. Infatti, una volta riaddestrato un modello sui nuovi dati, il modello non conterrà più informazioni sui dati rimossi.

- *Attack-based*: Dato che l'unlearning ha l'obiettivo principale di rimuovere leak di informazioni sensibili causati dall'over-fitting, un metodo si può ovviamente basare sugli attacchi volti a verificare le operazioni di unlearning.

- *Relearning time-based*: Anche il tempo con cui il modello riaddestrato ci mette per recuperare le performance originali può essere un fattore per verificare che ci sia ancora traccia dei informazioni sui sample dimenticati.

- *Accuracy-based*: Controlla le performance del modello dopo il processo di unlearning. Quindi, l'accuratezza sui dati dimenticati in linea di massima deve diminuire (come quella di un modello che non ha mai visto quei dati durante il training). L'accuratezza sugli altri dati invece dovrebbe rimanere stabile. Infine, se l'unlearning è stato utilizzato per rimuovere dati adversarial, la verifica consiste nel controllare che le performance ritornano a livelli normali dopo la rimozione del disturbo.

Calcoli teorici

- *Theory-based*: Alcuni metodi forniscono definizioni di unlearning certificate, che affermano che il modello dopo l'unlearning non è distinguibile da un modello allenato sui dati rimanenti, partendo da zero.

- *Information bound-based*: Metrica per verificare l'efficienza degli schemi di unlearning, in cui viene misurato il limite superiore dell'informazione residua dei sample che devono essere dimenticati. Più è bassa questa informazione residua, meglio lavora lo schema di unlearning.

2.5 Applicazioni nella sicurezza

L'impatto del concept drift sulla sicurezza è stato ampiamente dimostrato da studi del settore, che evidenziano come l'efficacia dei classificatori tenda a deteriorarsi rapidamente nel tempo. Un esempio è quello effettuato sul framework TESSERACT [16], dove gli autori evidenziano come la valutazione dei modelli di sicurezza sia spesso falsata da un *temporal bias*: Effettuando suddivisioni casuali del dataset, mescolando dati vecchi con dati nuovi, si ottengono performance che non rispecchiano l'efficacia reale del modello.

Tuttavia, l'applicazione di queste metodologie nei contesti NID presenta sfide uniche e maggiori complessità. Come sottolineato dal lavoro INSOMNIA [3], il traffico di rete non è solo un flusso continuo, ma è caratterizzato da pattern di drift estremamente rapidi dovuti a cambiamenti nelle configurazioni di rete, nuovi protocolli o aggiornamenti software. Nonostante ciò, la maggioranza della ricerca scientifica continua a privilegiare l'analisi dei malware rispetto ai NIDS. Tale disparità è riconducibile alla difficoltà intrinseca nel raccogliere, memorizzare e etichettare correttamente volumi massicci di traffico di rete che coprono archi temporali estesi. Mentre i file malevoli mantengono la loro natura nel tempo, il traffico di rete richiede un'etichettatura contestuale estremamente onerosa in termini computazionali ed umani.

Per far fronte a queste limitazioni, alcuni studi hanno proposto l'utilizzo di dataset sintetici o di breve durata, come il CICIDS2017 [19], che però falliscono nel catturare l'evoluzione dei dati nel lungo periodo.

Altri approcci, come il framework Transcend [12], tentano di identificare campioni che si discostano statisticamente dal training set originario per segnalare l'obsolescenza del modello, ma spesso faticano a scalare su throughput di rete elevati.

Infine, un ulteriore livello di complessità è introdotto dall'interazione tra il concept drift e tecniche adversarial sul Machine Learning. Come esplorato da Apruzzese et al. [4], il decadimento temporale può agire amplificatore per gli attacchi informatici. Gli studi dimostrano come perturbazioni intenzionali dei pacchetti, combinate con una conoscenza di un modello obsoleta, possano compromettere ulteriormente la resilienza dei NIDS.

Questa tesi si distingue per i seguenti contributi:

- A differenza di framework consolidati come TESSERACT [16] focalizzati su file statici, l'analisi qui viene applicata al dominio dinamico della rete.

- La sperimentazione utilizza il dataset MCFP su un arco temporale di 12 mesi, superando i limiti di lavori come INSOMNIA [3] o l'uso di dataset a breve termine, permettendo una valutazione realistica dell'utilità dei modelli nel lungo termine.

- Sebbene lavori come AISec '24 [4] introducano lo scenario adversarial combinato al drift, essi spesso adottano una distinzione netta tra blocco passato e futuro. Questa ricerca monitora

invece il sistema in modo granulare e sequenziale, mese per mese, integrando strategie di *Machine Unlearning* per una gestione proattiva dell'obsolescenza.

3. Paradigma metodologico per la resilienza al drift tramite CL e MU

In questo capitolo viene descritta la metodologia proposta per contrastare il fenomeno del concept drift nei sistemi di Network Intrusion Detection (NID). L'obiettivo principale è mitigare il decadimento prestazionale dei classificatori preservando, al contempo, l'integrità e l'attualità della conoscenza appresa.

A tal fine, si propone un framework basato sulla combinazione di tre componenti principali:

- *Experience Replay*: Per gestire l'apprendimento incrementale e contrastare il catastrophic forgetting attraverso il riutilizzo di campioni storici significativi.
- *Modular Network*: Un approccio architetturale che frammenta la conoscenza in moduli indipendenti, facilitando un aggiornamento mirato del sistema senza compromettere l'intera struttura.
- *Data Pruning*: una tecnica di Machine Unlearning che, traendo ispirazione dai principi Sharded, Isolated, Sliced, and Aggregated (SISA) [6], permette la rimozione selettiva di dati obsoleti o inquinati dal drift, garantendo una manutenzione efficiente e dinamica dei modelli.

L'integrazione di queste strategie definisce un sistema capace non solo di adattarsi a nuove minacce, ma anche di dimenticare attivamente dati non più rappresentativi del traffico di rete.

La scelta di utilizzare *Continual Learning* e *Machine Unlearning* in modo combinato risponde ai limiti evidenti delle attuali letterature. Mentre soluzioni come Transcend [12] si limitano a segnalare l'invecchiamento di un modello, e dataset standard come CICIDS2017 [19] non offrono una base temporale per l'adattamento, il nostro framework permette una difesa dinamica. Il *Continual Learning* fornisce la plasticità necessaria per apprendere nuove minacce, mentre il *Machine Unlearning* garantisce la stabilità del sistema, rimuovendo i modelli obsoleti o potenzialmente compromessi da dati manipolati, superando il problema di un accumulo indiscriminato di informazione tipico dei modelli di retraining periodico [4], oltre che a mantenere le risorse computazionali inferiori.

3.1 Threat Model

Il framework proposto si colloca a protezione di una rete aziendale di dimensioni medio-grandi (tipicamente una rete di "Classe B", con un numero di host compreso tra 256 e 65.000). Il traffico di rete fluisce tra la rete interna dell'organizzazione e l'Internet pubblico attraverso un *Border Router*.

In questa posizione strategica è integrato un *ML-NIDS* (Network Intrusion Detection System basato su machine Learning) che analizza i flussi di dati sotto forma di NetFlow in ingresso e in uscita.

In questo contesto, il traffico di rete viene catturato e processato in tempo reale. Le caratteristiche dei NetFlow vengono quindi estratte per alimentare i modelli di classificazione, i quali devono distinguere il traffico benevolo dai tentativi di intrusione. Tuttavia, data la natura dinamica dei protocolli e delle minacce, l'architettura non è statica ma prevede un ciclo di feedback per l'aggiornamento dei modelli, necessario a contrastare il *Concept Drift*.

3.1.1 Obiettivi del difensore

A differenza di approcci statici tradizionali, in questa sede il difensore assume un ruolo attivo nella manutenzione del sistema.

- *Obiettivo*: Il fine ultimo è la resilienza del modello nel lungo periodo. Il difensore deve massimizzare le performance nell'individuazione degli attacchi e mantenere bassa la quota di falsi positivi, impedendo che il *concept drift* renda obsoleti i *decision boundary* dei classificatori.

- *Conoscenza*: Il difensore ha accesso completo alla cronologia dei dati catturati, alle etichette fornite da analisi o sistemi di verifica, e ai parametri interni dei propri modelli.

- *Capacità*: A differenza dei sistemi statici descritti in letteratura (come CICIDS2017 [19] che coprono solo pochi giorni), il difensore in questa tesi implementa una strategia di manutenzione proattiva. Utilizza il *Continual Learning* (CL) per acquisire nuova conoscenza dai flussi di traffico recenti e il *Machine Unlearning* (MU) per rimuovere selettivamente i moduli o i dati che, a causa del drift o di potenziali iniezioni adversarial, degradano l'accuratezza globale.

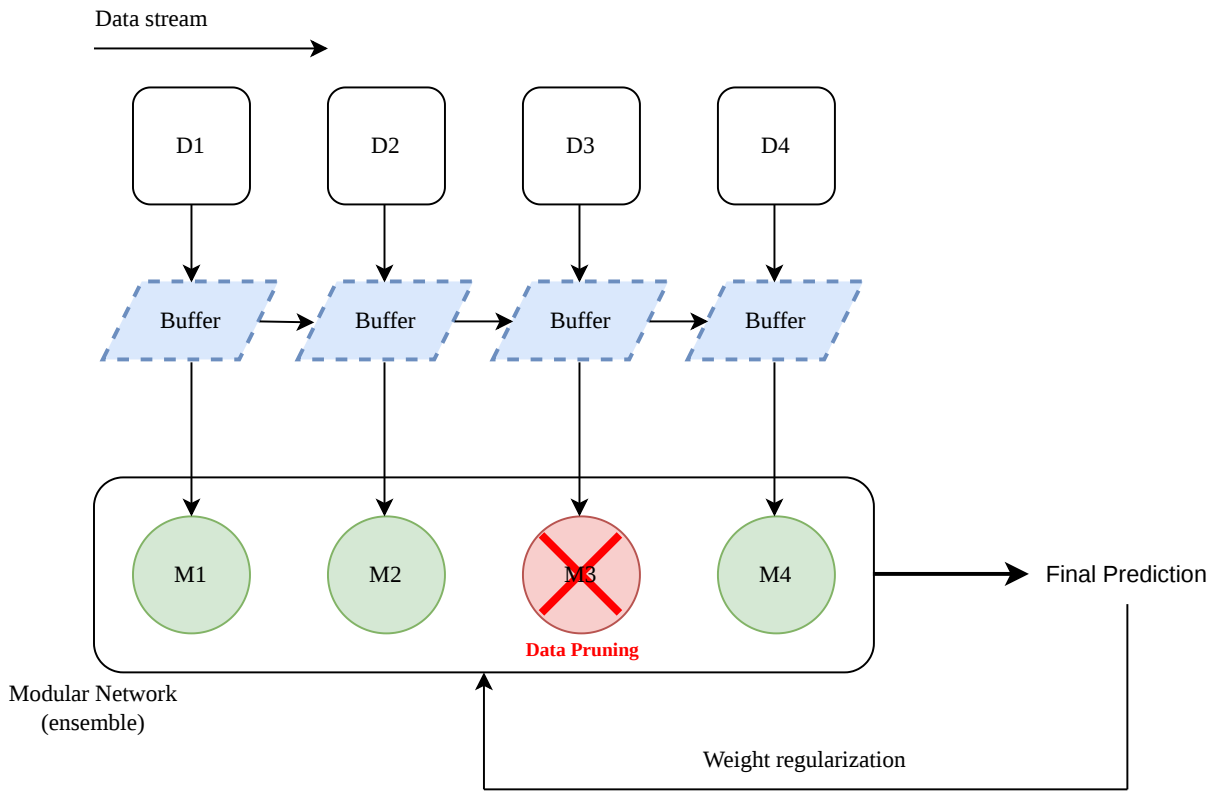

Figura 3.1: Rappresentazione schematica dell'approccio proposto: Integrazione tra CL e MU

3.2 Integrazione di Continual Learning e Machine Unlearning

L'obiettivo di questa ricerca è lo sviluppo di un framework capace di garantire la sostenibilità di un ML-NIDS nel lungo periodo, mitigando il *concept drift* senza incorrere nell'obsolescenza dei modelli o nel *catastrophic forgetting*. Il sistema proposto integra sinergicamente tecniche di Continual Learning (*Experience Replay* e *Modular Network*) e di Machine Unlearning (*Data Pruning*), ottimizzando il bilanciamento tra plasticità e stabilità.

3.2.1 Obiettivi

Il framework proposto è stato progettato per superare i limiti dei sistemi di rilevamento statici, soddisfacendo quattro requisiti operativi fondamentali necessari per l'impiego di modelli di Machine Learning in contesti di rete reali:

- *Resilienza al drift*: Obiettivo principale del framework. Consiste nel contrastare il decadi-

mento temporale delle prestazioni causato dal cambiamento delle distribuzioni del traffico (*concept drift*). Attraverso l'integrazione dell'*Experience Replay*, il sistema è in grado di aggiornare la propria base di conoscenza senza la necessità di un intervento manuale costante.

- *Performance costanti*: Il framework mira a mantenere elevati livelli di accuratezza (in particolare la *Recall* o *True Positive Rate*) lungo l'intero arco temporale di test. A differenza dei metodi di solo retraining incrementale, l'uso del Machine Unlearning permette di eliminare modelli obsoleti, che potrebbero generare falsi positivi o mascherare nuove tipologie di attacco.

- *Scalabilità*: L'architettura basata su *Modular Network* garantisce che il sistema possa crescere nel tempo senza diventare computazionalmente insostenibile. Invece di addestrare un unico modello monolitico su un dataset storico sempre più vasto, il framework aggiunge piccoli moduli indipendenti, rendendo il processo di aggiornamento rapido e adatto a volumi di dati tipici delle reti di Classe B.

- *Gestione delle risorse*: A differenza dei classici sistemi di ensemble che accumulano modelli indefinitamente, il framework integra una strategia di *Data Pruning* selettivo ispirata ai principi SISA [6]. Questa componente di Machine Unlearning permette di mantenere costante il carico computazionale durante la fase di classificazione: Identificando e rimuovendo i modelli meno rilevanti o obsoleti, il sistema impedisce l'espansione incontrollata della *Modular Network*. In questo modo, le risorse di calcolo vengono concentrate solo sulla conoscenza più efficace, garantendo che il NIDS operi entro i limiti hardware dell'infrastruttura aziendale senza degradare i tempi di risposta.

3.2.2 Scalabilità

Nei sistemi NIDS tradizionali, l'aggiornamento del modello avviene solitamente tramite il retraining di un unico classificatore monolitico su un dataset che cresce indefinitamente nel tempo. Questo approccio presenta due criticità in reti di grandi dimensioni:

- Costi computazionali elevati. Il tempo e le risorse necessarie per il retraining aumentano linearmente con la dimensione del dataset.

- *Catastrophic forgetting*: L'inserimento di nuovi dati in un modello già esistente rischia di alterare pesantemente i pesi precedentemente ottimizzati, causando la perdita di conoscenza pregressa.

Per superare questi limiti, il framework adotta un'architettura *Modular Network* ispirata ai principi di *sharding* e *isolation* del protocollo SISA [6]. Invece di far convergere tutta la conoscenza in un unico punto, il sistema frammenta l'apprendimento in una serie di modelli indipendenti e isolati tra loro.

A livello operativo, l'architettura si evolve in modo sequenziale seguendo la dinamica temporale dei dati di rete. Ogni nuovo ciclo di addestramento e testing genera un nuovo modello che nasce da zero in quel momento, addestrandosi esclusivamente sui dati correnti contenuti in un buffer (che verrà discusso nella sezione 3.2.3).

Poiché i modelli sono indipendenti tra loro, la nuova conoscenza acquisita non può fisicamente interferire con quella dei modelli precedenti, eliminando alla radice il problema del *catastrophic forgetting*.

Durante la fase di classificazione, i modelli operano come un ensemble, in cui ogni modello fornisce il proprio risultato e il sistema li aggrega per emettere un giudizio finale.

Vantaggio

Questa struttura garantisce che il sistema sia scalabile orizzontalmente. Infatti, per aggiornare il NIDS non serve riaddestrare anche sui dati passati, ma basta aggiungere un nuovo modello all'interno dell'ensemble. La complessità del sistema rimane così legata alla gestione dei singoli moduli e non alla massa critica dei dati storici accumulati che si avrebbe con un modello monolitico.

3.2.3 Resilienza del drift

Come ampiamente discusso precedentemente, il traffico di rete è per definizione non-stazionario: l'evoluzione naturale dei protocolli, degli aggiornamenti software e delle tattiche degli attaccanti genera il cosiddetto *Concept Drift*. In assenza di contromisure, questo fenomeno porta i confini di decisione del NIDS a diventare obsoleti, causando un aumento inaccettabile di falsi negativi e falsi positivi. Tuttavia, il tentativo di risolvere il concept drift tramite il Continual Learning introduce il

rischio di *catastrophic forgetting*: Il modello, nel tentativo di adattarsi ai dati più recenti, rischia di sovrascrivere la memoria di attacchi passati che sono ancora potenzialmente pericolosi.

Come descritto nella sezione 3.2.2, è stata scelta un'architettura *Modular Network*. Tale architettura è stata combinata ad una strategia di *Experience Replay*, che permette di selezionare selettivamente le esperienze passate e presenti. Invece di limitarsi ad un addestramento passivo sui dati correnti, l'*Experience Replay* garantisce che ogni nuovo modulo mantenga una visione bilanciata dell'ambiente di rete attraverso l'utilizzo di un buffer di memoria.

Il fulcro di questa strategia risiede nel buffer temporale, uno spazio di memoria controllato che serve a raccogliere i sample più significativi selezionati per massimizzare la resilienza del sistema. Questa selezione avviene nel seguente modo: Il buffer privilegia i sample che sono più ambigui per l'ensemble attuale, ovvero quelli che si trovano in prossimità del *decision boundary* (in pratica, con probabilità di classificazione vicina al 50%). Tali campioni sono più istruttivi per contrastare il drift, poiché rappresentano pattern non ancora consolidati.

Grazie a questa selezione, ogni nuovo modello della *Modular Network* riceve un set di informazioni utili, in modo da garantire un'evoluzione della conoscenza che preserva la capacità di riconoscere sia minacce note (stabilità) pur sempre adattandosi a quelle emergenti (plasticità).

3.2.4 Performance costanti

Tuttavia, l'introduzione di un'architettura basata su SISA (sharding) introduce una sfida critica per il sistema: Il traffico di rete, essendo naturalmente asimmetrico, ha un volume di dati benevolo che sovrasta drasticamente quello malevolo. Suddividendo i dati in piccoli buffer temporali isolati (shards), si corre il rischio che un modulo riceva una quantità di sample della classe minoritaria insufficiente per costruire un *decision boundary* valido. Inoltre, un attacco specifico potrebbe apparire in un solo shard, la cui eventuale rimozione comporterebbe la perdita totale di quella specifica capacità di difesa [13].

Per ovviare a questo limite, il framework interviene sulla composizione del buffer prima di ogni fase di retraining:

- Il sistema impone forzatamente un rapporto bilanciato tra sample benigni e malevoli all'interno del buffer.

- Se nel periodo corrente il traffico malevolo (o benigno) è insufficiente, il buffer non viene lasciato incompleto, ma viene integrato con i sample più significativi estratti nel periodo precedente. Questo garantisce che ogni modello dell'ensemble abbia una base di conoscenza robusta e non sia polarizzato verso la classe maggioritaria.

Mentre questo bilanciamento garantisce la qualità in ingresso, il Machine Unlearning ci viene in contro per la qualità in uscita. Infatti, in questa ricerca, l'unlearning non è un processo passivo, ma un meccanismo di manutenzione critico, basato sulla strategia di *Data Pruning*.

Al termine di ogni ciclo temporale, il framework controlla l'apporto reale di ogni singolo modello rispetto alla capacità di rilevamento complessiva dell'ensemble. Se un modulo, pur essendo addestrato su dati bilanciati, presenta un'accuratezza degradata a causa del *concept drift* o da potenziali iniezioni di dati *adversarial*, viene applicata una tecnica di *Data Pruning*. Invece, di tentare una correzione complessa, il framework elimina direttamente modello difettoso dall'ensemble.

Questo processo di potatura assicura che il *decision boundary* dell'ensemble sia definito solo dai modelli più affidabili e aggiornati, eliminando eventuali ambiguità statistiche introdotte da modelli obsoleti.

3.2.5 Gestione delle risorse

Un'altro problema in un sistema di Continual Learning basato su architettura modulare è l'aggiunta costante di nuovi modelli per far fronte al drift, generando un problema di saturazione. Infatti, ogni modello richiede banalmente dello spazio di archiviazione e dell RAM per i suoi pesi e parametri. Inoltre, poiché l'analisi del traffico avviene tramite un ensemble, ogni pacchetto deve essere processato da tutti i modelli attivi. Quindi, un numero eccessivo di modelli rallenterebbe troppo il monitoraggio, rendendolo inadatto a scenari di rete.

Il framework, applicando il *Data Pruning*, risolve anche questa criticità. Infatti non solo garantisce una migliore qualità delle performance, ma permette anche di agire da vincolo strutturale per la gestione delle risorse.

Il sistema è configurato per avere un numero minimo di modelli attivi. Dopodichè, ad ogni nuovo ciclo di retraining, il processo di unlearning identifica il modello meno utile basandosi sul

contributo delle metriche globali (*F1-Score*, *True Negative Rate* e *True Positive Rate*). Se un modulo dovesse avere un apporto negativo all'ensemble, verrebbe identificato e rimosso definitivamente.

Questo meccanismo garantisce che i requisiti computazionali (CPU e RAM) e i tempi di risposta del NIDS rimangano costanti nel tempo, indipendentemente dalla durata del deployment. Il sistema infatti dimentica fisicamente le porzioni di rete ormai inutili, permettendo al sistema di rimanere leggero e performante.

3.3 Formalizzazione dell'algoritmo

A completamento della descrizione metodologica, viene presentato lo pseudocodice di alto livello che sintetizza il funzionamento del framework proposto. L'algoritmo illustra l'interazione tra la gestione del buffer per l'Experience Replay, l'espansione della Modular Network e la successiva fase di Data Pruning per la manutenzione del sistema.

Algorithm 1 Ciclo di vita del Framework ML-NIDS proposto

1: **Fase 1: Pre-Deployment**
2: $D_{storico} \leftarrow$ Carica dati iniziali (T_0)
3: $Ensemble \leftarrow$ Addestra modulo iniziale M_0
4: $Buffer \leftarrow$ Inizializza con campioni rappresentativi per M_0 da $D_{storico}$
5: **Fase 2: Ciclo di retraining (per ogni mese T)**
6: **for** ogni nuovo shard temporale D_T **do**
7: $P \leftarrow$ Probabilità delle predizioni pesate dell'$Ensemble$ su D_T
8: *// Experience Replay*
9: $S_{critici} = S \in D_T : P_S = min(P)$ (≈ 0.5)
10: $Buffer.update(S_{critici})$
11: *// Data Pruning*
12: **if** $Ensemble.dimensione > threshold$ **then**
13: $M_{worst} \leftarrow$ Identifica modulo con minor contributo su metriche globali
14: **if** $Performance_{Ensemble} >= Performance_{(Ensemble-M_{worst})}$ **then**
15: $Ensemble.remove(M_{worst})$
16: **end if**
17: **end if**
18: *// Espansione Modular Network*
19: $M_T \leftarrow$ Modello addestrato dal $Buffer$ bilanciato
20: $Ensemble.append(M_T)$
21: **end for**

4. Realizzazione del framework: Integrazione dei processi di Replay e Pruning

4.1 Specifiche hardware e configurazione dell'ambiente di esecuzione

Per dare un contesto ai risultati ottenuti, in questa sezione vengono descritte le specifiche hardware e l'ambiente software adottati per lo sviluppo e il test del framework.

- **Risorse hardware**: I test sono stati eseguiti su un laptop personale utilizzato come workstation di sviluppo. Le specifiche tecniche del sistema utilizzato sono riassunte nella tabella 4.1

Componente	**Specifiche tecniche**
Modello	Huawei MateBook X Pro (2021)
Processore	Intel Core i7-1165G7
RAM	16 GB LPDDR4x
Archiviazione	SSD NVMe 1 TB
Sistema Operativo	Ubuntu 22.04.5 LTS

Tabella 4.1: Caratteristiche hardware della workstation utilizzata per il testbed.

- **Configurazione software**: L'architettura proposta è stata implementata interamente in linguaggio Python 3.10. Per la gestione dei modelli di Machine Learning è stata scelta la libreria *Scikit-learn*, in particolare per la sua implementazione efficiente con algoritmi basati su alberi decisionali. La manipolazione dei dati è stata gestita tramite la libreria *Pandas*. Per la fase di testing temporale, è stata utilizzata la libreria open source *Tesseract* [17], questa libreria offre funzionalità ottimizzate per gestire la divisione temporale dei dati, del training e del testing. Infine, si è deciso di utilizzare *Matplotlib* per la generazione dei grafici delle performance.

4.2 Dataset

Come dataset per testare il progetto, è stato scelto *MCFP (Malware Capture Facility Project)* [20]. Questo progetto è risultato particolarmente idoneo, in quanto si occupa di ottenere catture di dati di rete a lungo termine; a differenza di dataset statici, MCFP mantiene gli attacchi attivi per periodi prolungati, permettendo di raccogliere dati che coprono un arco temporale di poco meno di due anni (Settembre 2016 - Aprile 2018).

I dati sono stati acquisiti originariamente in formato PCAP. Per renderli processabili dai classificatori, sono successivamente stati convertiti in flussi bidirezionali (NetFlow) tramite il tool *Argus*, estraendo feature statistiche per ogni comunicazione.

In particolare, la scelta è stata di selezionare le catture separate di dati malevoli e dati benigni. Questa metodologia garantisce la presenza di etichette certe basate sull'origine del PCAP stesso, evitando incertezze derivanti da processi di labelling manuale o euristico su traffico misto.

La componente malevola del dataset è composta da flussi di rete generati principalmente da quattro famiglie di botnet: *Trickbot*, *Dridex*, *WannaCry* e *Artemis*. Tali minacce sono state selezionate per la loro eterogeneità comportamentale e per la disponibilità di catture in diversi mesi. Oltre a queste, è stata inclusa una categoria denominata "*Altri*", contenente campioni di altri malware secondari (es. trojan e backdoor minori), la cui inclusione è stata fondamentale per garantire la continuità dell'esperimento, assicurando la presenza di almeno una cattura di dati malevoli per ogni singolo shard temporale mensile, evitando così eventuali mancanze informative durante la fase di retraining del classificatore.

Al contrario, il traffico benigno è stato acquisito nella sua interezza dal portale MCFP poiché, in tale contesto, risulta numericamente inferiore rispetto al traffico malevolo. Questa scelta ha permesso di mantenere un rapporto il migliore possibile durante l'addestramento sequenziale.

la metodologia adottata prevede una gestione sequenziale dei dati organizzata in finestre temporali mensili. La scelta di una granularità mensile è motivata dalla necessità di disporre di una quantità di campioni sufficiente a garantire l'addestramento di modelli statisticamente significativi; ma anche per avere una divisione di slot temporali che coprisse almeno un anno di testing, in modo da visualizzare effettivamente l'andamento dei classificatori nel lungo termine.

La fase di training iniziale (pre-deployment) copre un intervallo di 8 mesi, specificatamente da

Settembre 2016 ad Aprile 2017. Questo blocco temporale funge da base di conoscenza consolidata, aggregando campioni benigni e malevoli per definire il *decision boundary* iniziale del sistema. A partire da Maggio 2017, il framework opera dinamicamente, trattando ogni mese successivo come un nuovo shard di test e fonte di aggiornamento tramite Continual Learning.

Periodo	**Fase**	**Tipologia Traffico**	**N. Pacchetti**	**N. Flow**
Set 2016 - Apr 2017	**Training** (8 mesi)	Benigno	9	111.141
		Malevolo (Dridex, Trickbot)	6	845.542
Maggio 2017	**Test 01**	Benigno	7	362.584
		Malevolo (WannaCry, Dridex, Trickbot)	9	290.193
Giugno 2017	**Test 02**	Benigno*		
		Malevolo (Artemis, Dridex, Trickbot, WannaCry)	8	296.936
Luglio 2017	**Test 03**	Benigno	2	6.257
		Malevolo (WannaCry)	9	101.176
Agosto 2017	**Test 04**	Benigno*		
		Malevolo (Artemis)	4	868.376
Settembre 2017	**Test 05**	Benigno	1	36.780
		Malevolo (Altri)	1	1.242.696
Ottobre 2017	**Test 06**	Benigno*		
		Malevolo (Altri)	1	1.687.825
Novembre 2017	**Test 07**	Benigno*		
		Malevolo (Altri)	1	1.288.826
Dicembre 2017	**Test 08**	Benigno*		
		Malevolo (Altri)	1	1.425.580
Gennaio 2018	**Test 09**	Benigno*		
		Malevolo (Dridex, Trickbot)	4	4.007.210
Febbraio 2018	**Test 10**	Benigno*		
		Malevolo (Altri)	2	195.606
Marzo 2018	**Test 11**	Benigno*		
		Malevolo (Trickbot, Altri)	2	1.343.176
Aprile 2018	**Test 12**	Benigno	1	4989
		Malevolo (Dridex)	1	966.349

Tabella 4.2: Distribuzione temporale del dataset MCFP: Volumi di traffico e partizionamento Train/Test.

** In assenza di catture native, il traffico benigno è integrato dal buffer di memoria per mantenere il bilanciamento 1:1.*

4.2.1 Preprocessing

Il preprocessing dei dati elencati nella Tabella 4.2 è stato eseguito in modo sequenziale per ogni singolo file PCAP tramite una pipeline automatizzata in ambiente *Bash*. Ogni file PCAP è stato processato individualmente per generare il corrispondente file CSV (Comme-Separated Values). Solo in un secondo momento, i record estratti sono stati aggregati in dataset mensili per permettere l'analisi del concept drift.

Per la conversione del traffico di rete da PCAP a CSV (Comma-Separated Values), è stato scelto di utilizzare *Argus*. Argus è uno strumento di auditing per dati di rete progettato per categorizzare e tracciare i pacchetti, raggruppandoli per flussi di dati specifici per protocollo. Dal punto di vista operativo, Argus è in grado di leggere informazioni da diverse sorgenti, tra cui binari generati da *tcpdump* o *libpcap*. La scelta di Argus come flow exporter è stata preferita ad altre soluzioni per la sua capacità di generare un set esteso di feature statistiche (più di trenta parametri), che includono metriche importanti per l'identificazione di botnet nel traffico di rete, come per esempio il carico di rete (*Load*) o il tasso di trasmissione (*Rate*), offrendo una granularità informativa superiore rispetto ad altri protocolli NetFlow.

Codice 4.1: Conversione di tutti i file `.pcap` in file `.argus` nella directory corrente

```
for file in <directory_path>/*.pcap
do
    argus -F argus.conf -r "\$file" -w "\${file\%.pcap}.argus"
done
```

Questi dati sono stati successivamente elaborati tramite il comando *ra*, che ha permesso di convertire i record di audit binari nel formato tabulare CSV necessario per l'addestramento dei modelli di Machine Learning.

Codice 4.2: Conversione di tutti i file `.argus` in file `.csv` nella directory corrente

```
for file in <directory_path>/*.argus
do
    ra -F ra.conf -r "$file" > "${file%.argus}.csv"
done
```

Questo comando è stato eseguito seguendo una configurazione definita nel file `ra.conf`. Il fulcro di tale configurazione risiede nella specifica `RA_FIELD_SPECIFIER`, che definisce l'elenco e l'ordine delle feature estratte per ogni flusso. Per garantire che il modello possa catturare la complessità del traffico botnet sono stati selezionati vari campi, qui raggruppati per tipologia logica:

- **Identificativi e Direzione:** `saddr` e `daddr` (indirizzi IP), `sport` e `dport` (porte logiche), `proto` (protocollo), `dir` (direzione del flusso) e `state` (stato della connessione).
- **Metriche Temporali:** `stime` (tempo di inizio), `dur` (durata totale), `sdur` (durata sorgente), `ddur` (durata destinazione), `avgdur`, `mindur` e `maxdur`.
- **Volumi di Traffico:** `pkts`, `spkts`, `dpkts` (conteggio pacchetti), `bytes`, `sbytes`, `dbytes` (byte trasferiti), `appbytes`, `sappbytes` e `dappbytes`.
- **Carico e Frequenza (Rate):** `load`, `sload`, `dload` (bit al secondo), `rate`, `srate` e `drate` (pacchetti al secondo).
- **Indicatori di Qualità (TCP/IP):** `loss`, `sloss`, `dloss` (pacchetti persi), `ploss` (percentuale di perdita), `jit`, `sjit`, `djit` (jitter), `tcprtt`, `synack` e `ackdat` (latenze del handshake TCP).
- **Intervalli (Inter-arrival Times):** `sintpkt`, `dintpkt` (tempo tra pacchetti), inclusi i valori di attività (`act`), inattività (`idl`), massimi e minimi per ciascuna fase (`sintpktactmax`, `sintpktactmin`, ecc.).
- **Dimensioni dei Pacchetti:** `smaxsz`, `sminsz`, `dmaxsz`, `dminsz` (dimensioni massime e minime dei segmenti trasmessi dalla sorgente e dalla destinazione).
- **Metadati e Flag:** `flgs` (flag di rete), `stos`, `dtos` (Type of Service), `sttl`, `dttl` (Time to Live), `shops`, `dhops` (hop di rete) e `swin`, `dwin` (finestre di ricezione TCP).

È importante specificare che i record NetFlow generati da *Argus* non possiedono un'etichetta nativa all'interno del traffico grezzo. Il processo di labelling è stato quindi eseguito in fase di estrazione sfruttando la natura controllata del dataset MCFP: poiché le catture malevole e benigne sono fornite in file PCAP separati e certificati all'origine, è stato possibile assegnare in modo

deterministico l'etichetta 0 (benigno) o 1 (malevolo) a ciascun flusso basandosi esclusivamente sul file sorgente di provenienza.

Dopo l'estrazione dei dati, è stata effettuata una fase di manipolazione e pulizia dei dati, per gestire valori mancanti, anomalie e migliorare le performance del modello.

Encoding dei numeri di porta dei pacchetti

Come prima cosa, sono stati gestiti i numeri di porta relativi alle feature `sport` (porta sorgente) e `dport` (porta destinazione). I valori nulli (*NaN*) o espressi in formato esadecimale sono stati mappati ad un valore convenzionale (-1).

Successivamente, le porte sono state raggruppate in quattro classi. Questa scelta segue la metodologia proposta in AISec '24 [4]. L'obiettivo di questa astrazione è duplice: Da un lato evitare che il modello debba gestre oltre 65.000 valori distinti, prevenendo l'overfitting su numeri di porta specifici. Dall'altro è utile per catturare la semantica del servizio, spostando l'attenzione del modello dal numero di porta alla funzione della connessione.

Le classi sono state definite in base agli standard IANA (Internet Assigned Numbers Authority):

- Porta non identificata (-1)
- Well-known (0-1023)
- Registered (1024-49151)
- Dynamic/Private (49512-65535)

Pulizia valori anomali

Successivamente, tutti i restanti valori nulli (*NaN*) sono stati sostituiti con il valore medio della colonna di appartenenza. Sono anche stati gestiti i valori infiniti, sostituiti con il valore massimo registrato per quella feature.

Un'operazione fondamentale ha riguardato la rimozione completa di tutti i record in cui il campo `Dir` (Direction) risultava nullo. Questa scelta è dettata dalla semantica stessa dei dati di rete: la direzione indica la natura della conversazione tra i due nodi (ad esempio se il flusso è unidirezionale

o bidirezionale). Senza questa informazione, il modello non sarebbe in grado di distinguere chi ha avviato la connessione e chi sta rispondendo.

Una volta garantita l'integrità dei flussi attraverso questa pulizia, i campi testuali rimanenti (incluso Dir) sono stati inoltrati alla fase di encoding per la conversione in formato numerico, come descritto nella sezione successiva.

Encoding di feature categoriali

Il dataset contiene altre variabili non numeriche fondamentali per la caratterizzazione del traffico di rete: Lo stato della connessione (`State`), i flag del protocollo (`Flgs`), il protocollo utilizzato (`Proto`) e la direzione della connessione (`Dir`). Poiché i modelli di Machine Learning richiedono input puramente numerici, è stato necessario applicare un processo di *Label Encoding* a queste feature. Il procedimento implementato ha seguito la seguente logica:

1. Per ogni colonna appartenente ad una feature categoriale, il sistema identifica tutti i valori distinti associati e assegna a ciascuno un indice intero partendo da 0.

2. Successivamente, sono state create nuove colonne (una per feature) contenenti i valori numerici corrispondenti.

3. Infine, le colonne testuali originali sono state rimosse per ridurre la ridondanza e l'occupazione di memoria, mantenendo solo le versioni numeriche necessarie per il training.

Questa tecnica permette di preservare l'informazione contenuta in feature complesse come i flag TCP (es. S_, SA_) o i protocolli (es. tcp, udp, icmp), rendendole variabili discrete che il modello può utilizzare per discriminare i comportamenti malevoli.

Encoding indirizzi IP

Infine, sono stati mappati gli indirizzi IP, dividendoli per rete interna e rete pubblica (Internet).

Questa modifica è di vitale importanza per garantire la capacità di generalizzazione del modello. Infatti, senza questo encoding il modello tenderebbe all'overfitting, imparando a memoria se un indirizzo IP è malevolo o no invece di identificare i comportamenti sospetti.

Riepilogo del dataset finale

Al termine delle operazioni di pulizia ed encoding, il dataset è composto da un totale di 39 colonne, di cui 37 feature, la label e la data. Le feature finali utilizzate per il training e il testing dell'architettura sono le seguenti:

- **Metriche Temporali e Statistiche:** `Dur`, `SrcDur`, `DstDur`, `Rate`, `SrcRate`, `DstRate`, `TcpRtt`, `SynAck`, `AckDat`.
- **Metriche di Volume e Carico:** `TotPkts`, `SrcPkts`, `DstPkts`, `TotBytes`, `SrcBytes`, `DstBytes`, `TotAppByte`, `SAppBytes`, `DAppBytes`, `Load`, `SrcLoad`, `DstLoad`.
- **Caratteristiche del Pacchetto e Protocollo:** `sTos`, `dTos`, `dTtl`, `sMaxPktSz`, `sMinPktSz`, `dMaxPktSz`, `dMinPktSz`, `Loss`, `DstTCPBase`, `SrcTCPBase`.
- **Feature Categoriali (Encoded):** `Sport_type`, `Dport_type`, `Flgs-f`, `Dir-f`, `Proto-f`, `State-f`.
- **Network Context:** `SrcPrivateIP`, `DstPrivateIP`.

È possibile visualizzare il numero finale di flussi di dati beigni e malevoli per ogni mese, ottenuti dopo la fase di preprocessing nella Tabella 4.4.

4.3 Classificatori

L'approccio adottato per la risoluzione al problema del concept drift è stato pensato per essere indipendente rispetto al classificatore adottato. Tuttavia, per validarne l'efficacia e la versatilità, sono stati selezionati tre algoritmi differenti in tre scenari distinti: *Random Forest (RF)*, *Logistic Regression (LR)* e *Multi-Layer Perceptron (MLP)*.

- **Random Forest**: Il Random Forest (RF) è stato selezionato come classificatore primario in quanto rappresenta uno degli algoritmi più efficaci e utilizzati nell'ambito dei ML-NIDS per la classificazione del traffico di rete. La sua architettura, basata su un ensemble di alberi decisionali, lo rende particolarmente idoneo a operare come nel rilevamento delle minacce per diversi motivi critici in ambito cybersecurity [26].

Periodo	Fase	Benevoli (0)	Malevoli (1)	Totale Mese
Sett 16 - Apr 17	**Pre-deployment**	111.123	453.893	565.016
Maggio 2017	**Test 01**	362.570	144.829	507.399
Giugno 2017	**Test 02**	0*	147.906	147.906
Luglio 2017	**Test 03**	6.253	50.792	57.045
Agosto 2017	**Test 04**	0*	434.754	434.754
Settembre 2017	**Test 05**	36.778	621.504	658.282
Ottobre 2017	**Test 06**	0*	843.762	843.762
Novembre 2017	**Test 07**	0*	644.441	644.441
Dicembre 2017	**Test 08**	0*	712.309	712.309
Gennaio 2018	**Test 09**	0*	2.003.322	2.003.322
Febbraio 2018	**Test 10**	0*	97.527	97.527
Marzo 2018	**Test 11**	0*	672.065	672.065
Aprile 2018	**Test 12**	4.987	482.901	487.888
Totale		**521.711**	**7.310.004**	**7.831.715**

Tabella 4.4: Distribuzione mensile dei flussi (Flow) estratti dal dataset MCFP post-preprocessing.

** In assenza di campioni benevoli nativi nel mese, il sistema utilizza i campioni critici conservati nel buffer di memoria per il retraining dell'ensemble.*

Una caratteristica fondamentale è la sua resilienza all'alta dimensionalità, permettendoci di mantenere un numero elevato di feature estratte. Inoltre, il RF è robusto di natura al rumore, tendendo ad ignorare le feature meno informative e focalizzandosi sui pattern comportamentali più significativi delle botnet.

- **Logistic Regression**: È stato scelta anche la Logistic Regression per verificare che le performance dell'algoritmo proposto fossero efficaci anche con modelli basati sull'ottimizzazione del gradiente, che operano in modo diverso rispetto ad algoritmi basati su alberi.

- **Multi-Layer Perceptron**: Consiste in una rete neurale composta da più strati di neuroni con funzioni di attivazione non lineari. È stato scelto per verificare le performance anche con un modello di *Deep Learning*.

Tutti i classificatori operano secondo un paradigma di apprendimento supervisionato (*Supervised Learning*). Il sistema utilizza le etichette assegnate durante la fase di preprocessing per apprendere la distinzione tra traffico benigno e malevolo.

Il problema viene quindi trattato come una classificazione binaria, definendo la Classe 0 come il traffico benigno (traffico di rete normale) e la Classe 1 come traffico malevolo (Botnet/Malware).

La particolarità è che non viene utilizzato un modello unico, ma un ensemble dinamico. Infatti, in ogni mese, la strategia di Continual Learning genera un nuovo classificatore che si aggiunge ai precedenti. La predizione finale sarà quindi una votazione pesata delle classificazioni di tutti i modelli dell'ensemble.

Per ognuno di questi modelli è stata effettuata una fase di configurazione e fine-tuning degli iperparametri specifica. I dettagli tecnici relativi ai parametri selezionati e alle procedure di regolarizzazione adottate sono descritti approfonditamente nella sezione 4.3.1.

4.3.1 Configurazione dei classificatori

Come introdotto precedentemente, per questo esperimento sono stati scelti tre classificatori: Random Forest (RF), Logistic Regression (LR) e Multi-Layer Perceptron (MLP). Per ognuno di essi sono state scelte le seguenti configurazioni:

1. *Random Forest*

Codice 4.3: Inizializzazione del classificatore Random Forest

```
clf = RandomForestClassifier(n_estimators=100, n_jobs=-1,
    random_state=42)
```

Il primo classificatore implementato è il Random Forest, la sua configurazione prevede 100 alberi decisionali, distribuiti su tutti i core della CPU disponibili, per accelerare la fase di addestramento. Infine, è stata impostato un seed di casualità fisso per garantire la riproducibilità dell'esperimento, in modo da avere un confronto equo tra la fase di training statico e quella di Continual Learning e Machine Unlearning.

2. *Logistic Regression*

Codice 4.4: Inizializzazione del classificatore Logistic Regression con standardizzazione delle feature e pesatura delle classi.

```
        clf = LogisticRegression(C=0.01, class_weight={0: 5, 1: 1},
            random_state=42)
        scaler = StandardScaler()

```

```
        X_train_past = scaler.fit_transform(X_train_past)
        X_test_past = scaler.transform(X_test_past)
```

In questo caso, la configurazione è stata più particolare, perché il modello presentava una forte tendenza all'overfitting. L'applicazione di una regolarizzazione molto stringente ($C = 0.01$) è servita per ridurre l'impatto di singole feature rumorose, in modo da apprendere meglio i pattern più generali e significativi. Inoltre, per correggere il modello dal classificare tutto il traffico benigno come malevolo, è stata introdotta una pesatura, assegnando un'importanza maggiore alla classe benigna (0) per ridurre l'impatto dei falsi positivi e migliorando la precisione complessiva.

3. *Multi-Layer Perceptron*

Codice 4.5: Inizializzazione del modello Multi-Layer Perceptron con architettura a due strati nascosti e standardizzazione delle feature.

```
        clf = MLPClassifier(hidden_layer_sizes=(25,25), batch_size=10,
            max_iter=250, alpha=0.1, random_state=42)
        scaler = StandardScaler()

        X_train_past = scaler.fit_transform(X_train_past)
        X_test_past = scaler.transform(X_test_past)
```

MLP è stato scelto per avere anche tra i test una tipologia di rete neurale. È stata scelta una struttura a due strati nascosti, con 25 neuroni ciascuno. Inoltre, è stato utilizzato l'iperparametro `alpha=0.1`, corrispondente al termine di regolarizzazione, utilizzato per prevenire l'overfitting dei pesi.

Per MLP è necessario anche applicare una fase di *Feature Scaling*. Nelle reti neurali, questa procedura è fondamentale in quanto estremamente sensibili alla scala numerica delle variabili in input.

Senza una corretta standardizzazione, le feature con valori di ordini di grandezza molto elevati rispetto alle altre tenderebbero a dominare il calcolo della funzione di costo, portando il modello a trascurare le altre feature (potenzialmente più informative).

4.4 Implementazione

L'implementazione[1] della metodologia proposta si suddivide in due fasi principali. La prima consiste nella configurazione di una parte di training iniziale (pre-deployment), in modo da avere un modello performante sui dati iniziali. La seconda invece, si tratta della gestione dinamica della conoscenza del classificatore attraverso tecniche di Continual Learning e Machine Unlearning.

- Modular Network (per il CL): Il sistema non utilizza un unico modello monolitico, ma una struttura a rete modulare basata su un ensemble incrementale. Ogni mese, viene aggiunto un nuovo modello nell'ensemble (addestrato sui dati correnti) che contribuisce alla decisione finale. Questo previene l'oblio catastrofico, poiché la vecchia conoscenza rimane preservata nei moduli precedenti mentre i nuovi catturano le variazioni del traffico.

- Experience Replay (per il CL): Per contrastare la scarsità di dati benevoli in determinati periodi, è stata implementata una strategia di Experience Replay. Utilizzando un buffer di memoria, il sistema "rigioca" o ripropone al modello campioni critici del passato. Questo garantisce che il classificatore non dimentichi le caratteristiche del traffico normale durante l'apprendimento di nuove minacce.

- Data Pruning (per il MU): Per mantenere l'efficienza del sistema e rimuovere informazioni ormai obsolete o fuorvianti a causa del concept drift, viene applicata una tecnica di Data Pruning (o Ensemble Pruning nel tuo caso specifico). Questa procedura identifica e rimuove i moduli o i dati che non contribuiscono più positivamente all'accuratezza globale, effettuando un vero e proprio "disapprendimento" della conoscenza non più valida.

A differenza di sistemi NIDS statici, l'architettura qui è progettata per operare in un ambiente sottoposto a cambiamenti costanti. Questo significa che il framework non verrà valutato su un singolo set di dati fisso, ma viene sottoposto ad un flusso sequenziale di dataset nel tempo. Questo viene fatto per misurare la capacità del sistema nel mantenere alte prestazioni nel lungo periodo, aggiornando la propria base di conoscenza nel tempo attraverso Continual Learning e ottimizzando la precisione del classificatore tramite operazioni di Machine Unlearning qualora i dati passati risultino obsoleti o dannosi.

[1] https://github.com/montaooo/Master-internship

Oltre ai benefici in termini di accuratezza, l'impiego del Machine Unlearning svolge un ruolo fondamentale anche nella scalabilità del sistema. Infatti, rimuovendo i modelli dell'ensemble non più rappresentativi o ridondanti, il MU permette di mantenere il sistema di dimensioni ridotte, risolvendo il problema di una crescità incontrollata della base di conoscenza. Questo approccio permette quindi di avere un risparmio significativo in termini di occupazione di memoria e risorse computazionali, assicurando che il NIDS rimanga efficiente anche dopo lunghi periodi di operatività.

La valutazione del framework è stata condotta analizzando la risposta del sistema a due diverse dinamiche di variazione del traffico. È opportuno sottolineare che il fenomeno del concept drift è considerato una componente sempre presente e ineludibile in qualsiasi ambiente di rete reale; l'evoluzione naturale dei servizi, degli aggiornamenti software e delle abitudini degli utenti produce un mutamento costante nelle distribuzioni statistiche dei dati.

L'esperimento prevede due macro-tipologie di traffico:

- **Traffico clean**: Per verificare l'efficacia dell'approccio combinato di Continual Learning e Machine Unlearning nel gestire il concept drift della rete, in assenza di attacchi adversarial.
- **Traffico adversarial**: Per verificare l'efficacia del sistema in presenza di attacchi adversarial basati sulla perturbazione delle feature.

4.4.1 Training

Questa prima fase riguarda la creazione del modello di riferimento, addestrato su un set di dati di rete iniziale rappresentativo. Questo modello servirà poi come punto di partenza per le successive iterazioni di Continual Learning.

A differenza della fase di test, per la quale sono stati utilizzati shard temporali mensili, il training set iniziale aggrega dati relativi al periodo tra Settembre 2016 e Aprile 2017. Questa scelta è principalmente dettata dalla necessità di raccogliere un volume di campioni benigni e malevoli sufficiente a garantire ottime performance. Nello specifico, il numero di campioni estratti in questa fase sono visualizzabili nella Tabella 4.4.

L'utilizzo di un intervallo temporale di 8 mesi per il training iniziale funge da base di conoscenza. Questo blocco rappresenta la conoscenza iniziale del sistema al momento della messa in produ-

zione, sulla quale poi verranno introdotte le tecniche di Experience Replay e Modular Network, in combinazione al Data Pruning per gli shard temporali successivi a partire da Maggio 2017.

Questa fase di pre-deployment garantisce che, a partire da Maggio 2017, il sistema non inizi il suo ciclo di vita "nel vuoto", ma possieda già una comprensione solida delle dinamiche di rete, pronta per essere messa alla prova dagli shard temporali successivi e dalle potenziali minacce adversarial.

Bilanciamento delle classi

Nei dataset di rete reali il traffico benigno è solitamente di gran lunga superiore rispetto a quello malevolo; tuttavia, nel dataset MCFP utilizzato in questa analisi, si osserva la dinamica opposta, con una netta prevalenza di flussi malevoli rispetto a quelli benigni. Per gestire questa distribuzione particolare e garantire che il *decision boundary* non venisse distorto dalla classe maggioritaria, è stato applicato un campionamento dei dati basato su un rapporto 1:1.

Codice 4.6: Downsampling della classe maggioritaria per ottenere un rapporto 1:1

```
1    X_train, y_train, t_train = spatial.downsample_set(X_train, y_train,
        t_train.values, min_pos_rate=1/2)
```

Per il campionamento, è stata utilizzata la funzione `downsample_set()` derivata dal repository ufficiale di Tesseract[2]. Tale funzione permette di applicare una tecnica di downsampling casuale. Matematicamente, il rateo 1:1 è stato raggiunto impostando il parametro `min_pos_rate` della funzione al valore 0.5.

Tale scelta deriva dalla definizione di frequenza relativa alla classe positiva (nel nostro caso, i dati relativi alle botnet) rispetto al totale dei sample.

$$\texttt{min_pos_rate} = \frac{n_{botnet}}{n_{normali} + n_{botnet}} = 0.5 \implies n_{normali} = n_{botnet} \tag{4.1}$$

Al termine delle operazioni di preprocessing, il volume dei flussi di dati estratti per la finestra di training iniziale (Settembre 2016 - Aprile 2017) ammontava a:

- Flussi benevoli: 111.123 record.

[2] https://github.com/s2labres/tesseract-ml-release

- Flussi malevoli: 453.893 record.

In linea con la strategia di bilanciamento 1:1 precedentemente descritta, la classe malevola è stata sottoposta ad un'operazione di downsampling per eguagliare il numero di sample benevoli disponibili. Di conseguenza, il dataset utilizzato effettivamente per il training iniziale, utilizzato per generare il primo modello della *Modular Network*, è composto da 222.246 flussi di dati, equamente spartiti tra benevoli e malevoli.

Questa base di conoscenza iniziale garantisce una rappresentazione equa di entrambe le classi, permettendo al sistema di definire un'accurata capcaità decisionale prima di procedere all'analisi sequenziale dei mesi successivi, dove la scarsità di dati benevoli (o malevoli in ambienti reali) renderà necessario l'intervento delle tecniche di Experience Replay tramite il buffer di memoria.

4.4.2 Experience Replay e Data Pruning nell'ensemble

Una volta stabilita la base di conoscenza iniziale, il sistema deve essere in grado di evolversi per contrastare il decadimento temporale dei modelli causato dal *concept drift*. In questa fase, l'architettura transita da un singolo classificatore a una *Modular Network*, in cui la conoscenza è distribuita su più modelli addestrati su diversi shard temporali.

Il cuore operativo di questa evoluzione è rappresentato dal meccanismo che gestisce i vari modelli. Infatti, il sistema non si limita ad aggiungere nuovi classificatori, ma coordina la loro attività attraverso un processo di predizione pesata. Tale approccio permette di trasformare l'ensemble in un sistema capace di auto-regolarsi, decidendo quali informazioni del passato conservare tramite l'*Experience Replay* e quali componenti rimuovere mediante il *Data Pruning*. In questo contesto, la predizione pesata non funge solo come decisione finale, ma costituisce la metrica principale per valutare l'incertezza dei dati e l'affidabilità dei singoli modelli.

Predizione pesata

Prima di addentrarsi nelle specifiche strategie di retraining e di unlearning, è necessario definire come il sistema aggrega le informazioni provenienti dai diversi modelli della *Modular Network*. A differenza di un approccio monolitico, questo framework opera con un ensemble di modelli.

Idealmente, l'obiettivo è quello di dare un maggiore peso ai modelli dell'ensemble che classificano correttamente il maggior numero di dati.

Invece di una classificazione binaria netta (0 o 1), il sistema interroga ogni modello tramite la funzione `predict_proba()`, che fornisce la probabilità che un dato di test ha di essere selezionato come benigno o malevolo da parte del modello corrente.

Codice 4.7: Classificazione probabilistica dei dati di test

```
def ensemble_predict_weighted(ensemble, X: pd.DataFrame, weights):
    if not ensemble:
        raise ValueError("Empty ensemble")

    all_probs = [model.predict_proba(X) for model in ensemble]
    weights = np.array(weights)
    weigthed_probs = np.array(all_probs) * weights[:, np.newaxis, np.newaxis]
    sum_probs = np.sum(weigthed_probs, axis=0)
    sum_weights = np.sum(weights)
    final_probs = sum_probs / sum_weights
    final_preds = np.argmax(final_probs, axis=1)

    return final_preds, all_probs, final_probs
```

In questo modo, ogni modello contribuisce alla probabilità finale in base ad un peso predefinito, che riflette la sua affidabilità osservata durante il training precedente. La probabilità finale viene calcolata come:

$$P_f(y|X) = \frac{\sum_{i=1}^{n}(w_i \cdot P_i(y|X))}{\sum_{i=1}^{n} w_i} \tag{4.2}$$

Questo garantisce che il calcolo del decision boundary sia influenzato maggiormente dai modelli migliori, cioè quelli con peso maggiore, rendendo la valutazione dell'importanza dei dati più accurata.

Questa modalità di predizione non è fine a se stessa, ma risulta utile anche per le fasi successive:

- Per il Continual Learning (Experience Replay), solo i dati su cui l'ensemble esprime un'alta incertezza (probabilità vicino al 50%) vengono selezionati per riempire il buffer di memoria, ottimizzando lo spazio e conservando i campioni più informativi.

- Per il Machine Unlearning (Data Pruning), il monitoraggio dei pesi e delle performance nella predizione pesata permette di individuare i modelli che non forniscono più un contributo utile, attivando la selezione e rimozione del modello dal sistema.

Selezione dei sample critici

Una volta ottenute le probabilità dell'ensemble, come prima cosa il sistema estrae dalle probabilità ottenute, il valore di probabilità più alto tra le classi.

Codice 4.8: Ottenimento del valore massimo tra le probabilità ottenute con `np.max()`

```
1    pred, all_probs, avg_probs = ensemble_predict_weighted(ensemble_models,
        X_test_past, weights)
2    max_probs = np.max(avg_probs, axis=1)
```

L'applicazione della funzione `np.max()` lungo l'asse delle classi permette di estrarre un vettore scalare con la maggiore confidenza per ogni sample. Matematicamente, i valori prossimi a 1 indicano una classificazione quasi certa. Al contrario, i valori più bassi corrisponderanno ai valori prossimi a 0.5, indicando i sample per i quali l'ensemble non riesce a stabilire una netta prevalenza di una classe sull'altra, ossia i valori più vicini al decision boundary. Tali valori sono i cosiddetti *borderline sample* (o ad alta incertezza) e rappresentano i dati più critici per la differenziazione tra traffico benevolo e malevolo. A differenza dei dati nelle zone ad alta confidenza (lontani dal decision boundary), i dati borderline sono considerati altamente informativi, in quanto definiscono la struttura stessa della funzione di decisione dell'ensemble.

Una volta ottenute le probabilità più alte, è stata utilizzata la funzione `np.argsort()` per ordinare i sample, partendo dai più bassi,

Codice 4.9: Selezione dei primi K campioni più vicini al *decision boundary*

```
1 def most_important_features(max_probs, X_test: pd.DataFrame, y_test: pd.
     DataFrame, K=10000):
2     idx_neg_full = np.where(y_test == 0)[0]
3     idx_pos_full = np.where(y_test == 1)[0]
4
5     neg_indexes = idx_neg_full[np.argsort(max_probs[idx_neg_full])[:K]]
6     pos_indexes = idx_pos_full[np.argsort(max_probs[idx_pos_full])[:K]]
```

```

    X_neg = X_test[neg_indexes]
    y_neg = y_test[neg_indexes]
    X_pos = X_test[pos_indexes]
    y_pos = y_test[pos_indexes]

    return X_neg, y_neg, X_pos, y_pos
```

In sintesi, per garantire la massima efficacia dell'*Experience Replay*, la selezione dei campioni per il buffer non avviene in modo casuale, ma segue una logica di campionamento bilanciato. Una volta ottenuti i punteggi di confidenza tramite l'ensemble, la funzione opera secondo i seguenti criteri:

- Innanzitutto, il sistema non si affida alla predizione del modello per suddividere i dati, ma utilizza la *ground-truth* per isolare tutti i campioni realmente benevoli da quelli realmente malevoli.
- Per ciascuna delle due categorie, il sistema applica la funzione `np.argsort()` sul vettore `max_probs`. Poiché l'ordinamento è crescente, i valori estratti corrispondono ai campioni con la confidenza più bassa (i valori più vicini a 0.5). Questi rappresentano i campioni più vicini al *decision boundary*, ovvero i dati per i quali l'ensemble ha la massima incertezza.
- Infine, per evitare sbilanciamenti che potrebbero favorire una classe rispetto all'altra, la funzione estrae separatamente i K sample più incerti per la classe benevola e i K sample più incerti per la classe malevola. In questo esperimento, $K = 50.000$ per ogni classe, determinando un inserimento totale di 100.000 nuovi campioni (50k + 50k) all'interno del buffer di memoria ad ogni iterazione.

Tale valore è stato ritenuto un compromesso ottimale, in quanto una base di 100.000 flussi critici è sufficientemente ampia da rappresentare la variabilità statistica del traffico di rete e la diversità dei pattern di attacco, senza però eccedere nelle risorse computazionali richieste durante le fasi di retraining mensile dell'ensemble.

Suddivisione temporale

A differenza dei metodi di testing tradizionali, questa metodologia prevede una gestione temporale e sequenziale dei dati. Infatti, il dataset MCFP è stato suddiviso per mesi, permettendo di simulare l'evoluzione del traffico di rete e la comparsa di nuove minacce nel tempo.

La valutazione procede il mese immediatamente successivo al training iniziale: Partendo da Maggio 2017, ogni mese viene trattato come un dataset indipendente in in input all'ensemble. La finestra temporale completa ricopre un intero anno, fino ad Aprile 2018.

Traffico 2017	**Apr**	**Mag**	**Giu**	**Lug**	**Ago**	**Set**	**Ott**	**Nov**	**Dic**
Traffico benigno	✓	✓		✓		✓			
Artemis			✓		✓				
Dridex	✓	✓	✓						
Trickbot	✓	✓	✓						
WannaCry		✓	✓	✓					
Others						✓	✓	✓	✓

Traffico 2018	**Gen**	**Feb**	**Mar**	**Apr**
Traffico benigno				✓
Artemis				
Dridex	✓			✓
Trickbot	✓		✓	
WannaCry				
Others		✓	✓	

Tabella 4.6: Matrice di occorrenza delle tipologie di traffico per ogni mese.

Continual Learning

La procedura di Continual Learning adottata viene eseguita nel seguente modo:

- Dopo la predizione dei NetFlow corrispondenti al mese corrente T_i, si calcola l'importanza dei smaple ottenuti per poi inserirne i migliori 50.000 nel buffer di memoria, come nell'inizializzazione iniziale.

  In particolare, il sistema implementa un meccanismo di auto-conservazione del buffer per contrastare l'eventuale scarsità di sample in determinati periodi di tempo. Se un mese

presenta un numero di dati benevoli o malevoli inferiore alla soglia K, si attiva una procedura di compensazione dinamica. Attingendo ai sample più informativi del buffer del mese precedente, il sistema garantisce che il training set per il nuovo modello dell'ensemble sia sempre bilanciato (1:1) e con un adeguato numero di dati (K).

Questa strategia permette di impedire che periodi di bassa attività di rete svuotino il buffer, e di conseguenza la capacità di generalizzazione dei modelli creati.

Codice 4.10: Utilizzo del buffer per compensare le mancanze di dati

```
1        X_test_neg, y_test_neg, X_test_pos, y_test_pos =
            most_important_features(max_probs, X_test, y_test, K=
            target_samples)
2
3        if len(y_test_neg) < target_samples:
4            diff = target_samples - len(y_test_neg)
5            X_test_neg = np.vstack([X_test_neg, buffer_X_neg[:diff]])
6            y_test_neg = np.concatenate([y_test_neg, buffer_y_neg[:diff
                ]])
7        if len(y_test_pos) < target_samples:
8            diff = target_samples - len(y_test_pos)
9            X_test_pos = np.vstack([X_test_pos, buffer_X_pos[:diff]])
10           y_test_pos = np.concatenate([y_test_pos, buffer_y_pos[:diff
                ]])
11
12       buffer_X_neg = X_test_neg
13       buffer_y_neg = y_test_neg
14       buffer_X_pos = X_test_pos
15       buffer_y_pos = y_test_pos
```

- Successivamente, avviene una fase di aggiustamento dei pesi in base alle predizioni del periodo attuale. La qualità dell'ensemble è garantita dai pesi, che devono essere adattivi in base all'andamento delle predizioni. Per questo motivo, invece di assegnare un'importanza statica ai modelli, il sistema valuta le prestazioni di ogni membro dell'ensemble sul mese corrente.

Codice 4.11: Funzione per il calcolo dei pesi dei vari modelli dell'ensemble

```python
def calculate_weights(y_test: pd.DataFrame, all_probs, botnet: str):
    weights = []
    for prob in all_probs:
        pred = np.argmax(prob, axis=1)
        tn, fp, fn, tp = confusion_matrix(y_test, pred, labels=[0,1])
            .ravel().tolist()
        if tp + fn == 0:
            tpr = 1
        else:
            tpr = tp / (tp + fn)

        if tn + fp == 0:
            tnr = 1
        else:
            tnr = tn / (tn + fp)
        weights.append(decimal_floor(np.sqrt(tpr * tnr), 3))

    return weights
```

Il calcolo delle performance dei modelli viene fatto come $\sqrt{TPR \cdot TNR}$, dove TPR sta per True Positive Rate, che misura la percentuale di veri positivi (malevoli) correttamente identificati; mentre TNR sta per True Negative Rate, che misura la percentuale di veri negativi (benigni) correttamente identificati. È stata scelta questa media geometrica per la sua capacità di penalizzare i modelli che si sbilanciano verso una singola classe. Infatti, se un modello dovesse perdere efficienza nel riconoscere le botnet o se cominciasse a generare troppi falsi positivi a causa del concept drift, il suo peso sarebbe automaticamente ridotto, privilegiando i modelli che mantengono un maggiore equilibrio.

- Infine, l'approccio di Continual Learning si completa con la generazione di un nuovo membro dell'ensemble:

Codice 4.12: Creazione di un nuovo modello addestrato sui dati nuovi

```python
X_new_ensemble = np.vstack([X_test_neg, X_test_pos])
y_new_ensemble = np.concatenate([y_test_neg, y_test_pos])

```

```
    ensemble_models.append(RandomForestClassifier(n_estimators=100,
        n_jobs=-1, random_state=42).fit(X_new_ensemble, y_new_ensemble))
```

Ad ogni mese, viene addestrato un nuovo modello (Random Forest in questo caso) sui sample più vicini al decision boundary, integrati nel buffer. Questo permette al sistema di ottenere nuove conoscenze in modo modulare (*Modular Network*): Invece che modificare modelli esistenti, andiamo ad aggiungere un nuovo componente specializzato sul traffico più recente.

Questo approccio risulta molto utile per evitare *Catastrophic Forgetting*, poiché la conoscenza pregressa rimane preservata nei membri più anziani dell'ensemble.

Si può facilmente notare che un approccio limitato a questa sola soluzione introduca a problematiche critiche per la scalabilità e l'affidabilità del sistema:

- *Complessità computazionale*: Considerando che ad ogni intervallo di tempo viene aggiunto un nuovo modello all'ensemble, il costo computazionale del sistema sia in fase di training che in fase di testing cresce linearmente.

- *Data poisoning*: Non tutti i dati nuovi sono effettivamente validi. Se il sistema dovesse apprendere del traffico contenente dati *adversarial* o pattern con del rumore, tale conoscenza andrebbe a degradare le performance future dell'ensemble.

- *Attacchi obsoleti*: Le tecniche di attacco si evolvono costantemente (come il traffico benigno, anche se più lentamente). Un modello addestrato su botnet di anni precedenti potrebbe conservare informazioni ormai non più rilevanti, agendo solo come "rumore di fondo" nel processo di votazione pesato dell'ensemble.

Proprio per risolvere queste criticità, l'architettura adotta anche una strategia di Machine Unlearning, come processo di ottimizzazione della memoria. Infatti, il sistema deve essere in grado di dimenticare i modelli che sono ridondanti, obsoleti o rumorosi, garantendo che l'ensemble rimanga sempre pulito e focalizzato correttamente.

Machine Unlearning

La strategia di Machine Unlearning utilizzata non si basa su una semplice rimozione dei dati più vecchi, ma su un'analisi del contributo di ogni singolo modello alle performance globali dell'ensemble.

Codice 4.13: Analisi del contributo dei vari modelli dell'ensemble

```
if len(ensemble_models) > 5:
    if i != 1:
        total_tpr = results['TPR'][-1]
        total_tnr = results['TNR'][-1]
        total_f1 = results['F1'][-1]

        total_tpr = 0 if type(total_tpr) == str else total_tpr
        total_tnr = 0 if type(total_tnr) == str else total_tnr
        total_f1 = 0 if type(total_f1) == str else total_f1

        for j in range(len(ensemble_models)):
            models_left = ensemble_models[:j] + ensemble_models[j+1:]
            weights_lef = weights[:j] + weights[j+1:]
            pred, all_probs, avg_probs = ensemble_predict_weighted(models_left
                , X_test, weights_lef)
            calculate_metrics(y_test, pred, unlearning, botnet)
```

Innanzitutto, questo processo non viene eseguito ad ogni iterazione, ma viene attivato da un vincolo di dimensionalità dell'ensemble, per l'esattezza con una dimensione minima di 6 modelli al suo interno. Questo viene fatto per evitare di avere troppi pochi modelli nell'ensemble durante le predizioni, rischiando di incorrere nel catastrophic forgetting.

Una volta attivato il trigger, il sistema, per ogni modello j nell'ensemble, simula la sua rimozione e ricalcola le performance ottenute con i restanti modelli.

Dopodiché vengono confrontate le prestazioni migliori (ottenute escludendo il modello con il contributo minore) con le performance dell'ensemble originale completo. Questo paragone permette di stabilire se la rimozione di un componente dell'ensemble vada effettivamente a migliorarne le prestazioni o se ne provochi un degrado.

Codice 4.14: Eventuale rimozione del modello in caso di superamento della soglia stabilita

```
bonus = 0.01
unlearning = {k: [1 if x == '/' else x for x in v] for k, v in unlearning.
    items()}
unlearning['F1'] = [el + bonus for el in unlearning['F1']]
unlearning['TNR'] = [el + bonus for el in unlearning['TNR']]
unlearning['TPR'] = [el + bonus for el in unlearning['TPR']]

max_index = unlearning['F1'].index(max(unlearning['F1']))

if unlearning['F1'][max_index] >= total_f1 and unlearning['TNR'][max_index
    ] >= total_tnr and unlearning['TPR'][max_index] >= total_tpr:
    del ensemble_models[max_index]
    del weights[max_index]
    print(f"Rimozione modello {max_index}")
```

La decisione finale di eliminazione è regolata con un criterio. Per evitare di rimuovere modelli ancora utili a causa di cali di performance minimi, è stato introdotto un *fattore di bonus* (0.01). Un modello viene rimosso solo se la sua esclusione porta F1, TPR e TNR a superare o eguagliare le performance originali dell'ensemble aumentate con il bonus. A livello matematico, il modello j viene rimosso se:

$$p_{ens-M_j} \geq p_{ens} + \delta \quad \forall \mathrm{p} \in F1, TPR, TNR \tag{4.3}$$

Questo approccio rende il Machine Unlearning uno strumento di potatura dei dati (*Data Pruning*). Infatti, se un modello dovesse risultare obsoleto o dovesse contenere dati anomali o dannosi per il sistema, verranno identificati e scartati.

4.5 Scenari di Test

Per verificare le prestazioni dell'architettura e delle strategie di Continual Learning e Machine Unlearning, la fase di testing è stata articolata in tre scenari. L'obiettivo è di visualizzare le performance di base e di misurare la resilienza dell'intero sistema a fronte di situazioni normali e di tentativi di evasione.

4.5.1 Pre-Deployment

Il primo scenario rappresenta la validazione iniziale del sistema. In questa fase viene testato il modello utilzzando una partizione di dati fissa relativa al periodo iniziale (Settembre 2016 - Aprile 2017). In questa fase, il dataset è stato suddiviso in training set e testing set secondo un rapporto 70:30.

I risultati ottenuti in questa fase servono a confermare che il sistema sia in grado di distinguere in modo efficiente i dati benevoli da quelli malevoli. Le prestazioni registrate in questo primo scenario serviranno nei successivi come termine di paragone per quantificare l'impatto del concept drift e degli attacchi mirati.

4.5.2 Valutazione su traffico clean

Questo scenario simula l'operatività del NIDS in un ambiente realistico nel corso di 12 mesi. Il termine "clean" indica che i dati analizzati appartengono al traffico di rete standard, senza alcun tipo di manipolazione o attacco di tipo adversarial.

La valutazione non viene eseguita in modo univoco, ma viene divisa in tre configurazioni, permettendo di visualizzare il contributo di ciascuna strategia implementata.

- *Baseline*: In questa prima casistica, il modello addestrato nella fase di *Pre-Deployment* (Aprile 2017), viene testato nei mesi successivi senza ricevere alcun aggiornamento.

  L'obiettivo è quello di misurare il degrado delle prestazioni dovuto al concept drift, aspettandoci quindi delle grosse oscillazioni o un calo generale delle performance. Questo caso serve per dimostrare quanto velocemente un NIDS tradizionale diventi obsoleto di fronte all'evoluzione delle minacce.

- *Solo Continual Learning*: Il sistema qui viene aggiornato mensilmente con i nuovi dati e con l'aggiunta di nuovi membri nell'ensemble, ma senza mai rimuovere la conoscenza pregressa.

  L'obiettivo è di verificare la capacità di adattamento del sistema a fronte del concept drift, ma anche di osservare l'accumulo indefinito dei modelli e di dati potenzialmente obsoleti che portino ad un inquinamento del sistema. L'aspettativa è quindi quella di visualizzare performance oscillanti nel tempo, in base all'evoluzione delle botnet nel tempo.

- *Continual Learning e Machine Unlearning*: Rappresenta la configurazione completa della metodologia proposta. Il sistema impara mensilmente dai nuovi dati e contemporaneamente applica la strategia di *Pruning* rimuovendo al massimo un modello per mese.

  L'obiettivo è di dimostrare che la rimozione selettiva dei modelli obsoleti o meno performanti non solo ottimizza le risorse computazionali, ma può effettivamente migliorare le performance complessive, rimuovendo il rumore di fondo.

In questi scenari, l'attenzione non è rivolta solo al punteggio assoluto delle metriche, ma alla loro stabilità nel tempo. Un sistema efficace deve riuscire a rimanere sempre a valori accettabili piuttosto che essere perfetto un mese e creare tanti falsi positivi il mese successivo.

4.5.3 Valutazione su traffico adversarial

L'ultimo scenario di test è progettato per valutare la resilienza dell'approccio adottato contro attacchi adversarial. In particolare, l'attenzione è focalizzata sulla differenza tra una configurazione con solo Continual Learning e una con Continual Learning e Machine Unlearning di fronte ad un *Poisoning Attack*.

Attacco

Per simulare un attacco di posioning realistico, è stata effettuata una manipolazione di tre feature critiche per il rilevamento del traffico di rete:

- *Dur*: Rappresenta l'intervallo temporale tra l'invio del primo pacchetto e l'ultimo pacchetto di un flusso.

  Le botnet spesso utilizzano comunicazioni molto brevi (necessitano di inviare comandi rapidi) o, al contrario, connessioni persistenti a lungo termine. Perturbare questa feature significa tentare di far apparire un attacco come una normale sessione di navigazione.

- *SrcBytes*: Indica la quantità di dati inviati dalla sorgente alla destinazione, calcolata come somma in bytes.

- *TotPkts*: Rappresenta il numero totale di pacchetti scambiati all'interno del flusso.

Dur	SrcBytes	Pkts
1	1	1
10	64	10
120	1024	120

Tabella 4.7: Valori perturbati per ogni feature utilizzati negli attacchi

Pacchetti e bytes definiscono la dimensione media dei pacchetti scambiati. Manipolare questi valori permette ad un attaccante di cammuffare la struttura dell'attacco, facendolo somigliare a comunicazioni standard.

L'attacco è stato strutturato iniettando il 100% dei dati malevoli per 3, 4 e 5 mesi consecutivi. L'obiettivo di questa strategia è di forzare il modello di Continual Learning a visualizzare dati perturbati, integrandoli progressivamente come parte della propria conoscenza consolidata.

Il confronto tra le due diverse configurazioni mira a validare l'ipotesi per cui un modello con solo Continual Learning subisca il degrado maggiore. Infatti, senza un meccanismo di controllo della qualità dei modelli acquisiti, il sistema incorporerà sicuramente i nuovi modelli addestrati su dati perturbati.

Per analizzare nel dettaglio la soglia di tolleranza dell'architettura proposta, l'iniezione di dati malevoli perturbati è stata ulteriormente suddivisa in sotto-scenari incrementali. Tre scenari verificano le prestazioni sulle singole feature attaccate, mentre uno comprende un attacco volto a tutte e tre le feature contemporaneamente. Ogni scenario modifica i valori delle feature secondo valori predefiniti, definiti nella Tabella 4.7.

Coerenza matematica

Un aspetto molto importante nell'implementazione dell'attacco adversarial è il mantenimento della coerenza interna del dataset. Poiché le feature di rete sono spesso legate da relazioni matematiche, la modifica isolata di un singolo parametro (come la durata) creerebbe sample irrealistici.

Per ovviare a questo problema, è stata progettata la funzione `poison_features()` per ricalcolare dinamicamente le feature derivate ogni volta che una delle feature base (durata, bytes e pacchetti) viene perturbata.

Codice 4.15: Funzione per l'avvelenamento delle feature

```python
def poison_features(X, y, X_columns: pd.Index, features: list, increments: list):
botnet_indexes = np.where(y == 1)[0]
feature_columns = [X_columns.get_loc(column) for column in features]
botnet_indexes = np.random.choice(botnet_indexes, int(len(botnet_indexes) * 1), replace=False)
idx = {name: X_columns.get_loc(name) for name in ["Dur", "SrcDur", "SrcBytes", "TotBytes", "TotPkts", "SrcPkts", "Rate", "SrcRate", "Load", "SrcLoad", "sMaxPktSz", "dMaxPktSz"]}

for c_index, i, f in zip(feature_columns, increments, features):
    X[botnet_indexes, c_index] += i

    if f == "Dur":
        X[botnet_indexes, idx['SrcDur']] += i
    elif f ==  "SrcBytes":
        X[botnet_indexes, idx['TotBytes']] += i
    elif f == "TotPkts":
        X[botnet_indexes, idx['SrcPkts']] += i


X[botnet_indexes, idx['Rate']] = np.divide(X[botnet_indexes, idx['TotPkts']]-1, X[botnet_indexes, idx['Dur']], out=X[botnet_indexes, idx['Rate']], where= X[botnet_indexes, idx['Dur']] != 0)
X[botnet_indexes, idx['SrcRate']] = np.divide(X[botnet_indexes, idx['SrcPkts']]-1, X[botnet_indexes, idx['SrcDur']], out=X[botnet_indexes, idx['SrcRate']], where=X[botnet_indexes, idx['SrcDur']] != 0)
X[botnet_indexes, idx['Load']] = np.divide((X[botnet_indexes, idx['TotBytes']] - (X[botnet_indexes, idx['sMaxPktSz']]+X[botnet_indexes, idx['dMaxPktSz']]))*8, X[botnet_indexes, idx['Dur']], out=X[botnet_indexes, idx['Load']], where=X[botnet_indexes, idx['Dur']] != 0)
X[botnet_indexes, idx['SrcLoad']] = np.divide((X[botnet_indexes, idx['SrcBytes']] - X[botnet_indexes, idx['sMaxPktSz']])*8, X[botnet_indexes, idx['SrcDur']], out=X[botnet_indexes, idx['SrcLoad']], where=X[botnet_indexes, idx['SrcDur']] != 0)
```

In questo modo, i dati adversarial iniettati nel ciclo di Continual Learning appaiono come

sessioni di traffico effettivamente plausibili dal punto di vista matematico.

5. Risultati

In questo capitolo verranno analizzati i risultati sperimentali ottenuti dall'architettura proposta. La valutazione ha due principali obiettivi: Da un lato, verificare la capacità del sistema di mantenere alte le prestazioni delle classificazioni nel tempo; dall'altro lato, verificare la resilienza del sistema di *Machine Unlearning* e *Continual Learning* di fronte al drift e ad attacchi adversarial.

Per fornire una visione completa dell'approccio adottato, l'analisi è stata strutturata in tre scenari:

1. *Pre-Deployment*: Scenario di base effettuato su dati statici, volto a valutare la capacità iniziale del modello di distinguere le minacce dal traffico normale.

2. *Clean*: Analizza il comportamento del sistema in un contesto di produzione simulato su un arco temporale di un anno. In questa fase si valuta l'impatto del *Concept Drift* e della capacità del sistema di gestirlo attraverso le meccaniche di *Continual Learning* e *Machine Unlearning*.

3. *Adversarial*: Costituisce lo scenario di stress per il sistema, in cui vengono introdotti degli attacchi con l'obiettivo di eludere il rilevamento. Qua viene misurata l'efficacia del *Machine Unlearning* nel rimuovere i modelli compromessi, rispetto ad un approccio di solo *Continual Learning*.

Inoltre, tutti i test sono stati iterati cinque volte, calcolandone la media, per avere dei valori di output affidabili e non esclusivamente dipendenti dalla prova corrente.

5.1 Descrizione delle metriche di valutazione

Il sistema di rilevamento delle intrusioni si basa su una *classificazione binaria*: L'algoritmo riceve in input un vettore di feature rappresentante un flusso di rete e deve assegnarlo ad una di due classi mutualmente esclusive:

- Classe negativa (0): Traffico benigno, ovvero le normali attività di rete.

- Classe positiva (1): Traffico malevolo, da identificare come minaccia.

Per valutare le prestazioni di una classificazione binaria, si utilizza la *confusion matrix*, una tabella che permette di confrontare le etichette reali dalle predizioni effettuate dal modello. Dalla combinazione di questi valori si ottengono quattro categorie di dati:

- *True Positive* (TP): Il modello ha predetto correttamente una minaccia.
- *True Negative* (TN): Il modello ha predetto correttamente il traffico benigno.
- *False Positive* (FP): Il modello ha segnalato un attacco dove in realtà non c'era (falso allarme). In ambito aziendale, un eccesso di falsi positivi causa un'*alert fatigue* degli operatori.
- *False Negative* (FN): Il modello ha classificato come benigno un attacco reale. Si tratta della situazione più pericolosa, poiché l'intrusione passa inosservata.

L'importanza di bilanciare queste quattro categorie risiede nelle ricadute pratiche sulla gestione della sicurezza. Come accennato poco fa, un elevato numero di falsi positivi è la causa principale dell'*alert fatigue*.

Questo fenomeno si verifica quando gli operatori del SOC, sommersi da una quantità ingestibile di correzioni errate, iniziano ad ignorare o a gestire con minore attenzione gli allarmi del sistema. In ambito aziendale, le conseguenze sono gravi, infatti la presenza di troppi segnali rende più complicato identificare un attacco reale quando effettivamente avviene. Pertanto, una delle sfide della classificazione è di mantenere alte le performance anche per garantire che ogni allarme generato dal NIDS sia effettivamente meritevole di un'indagine tecnica.

5.1.1 Metriche di valutazione

Per quantificare l'efficacia del sistema, sono state prese in considerazione le seguenti metriche:

- *Precision*: Consiste nella frazione di allarmi corretti rispetto al totale delle segnalazioni effettuate dal modello.

$$Precision = \frac{TP}{TP + FP}$$

- *Recall* o *True Positive Rate* (TPR): Misura la capacità del NIDS di rilevare correttamente tutte le attività malevole presenti nel traffico di rete. Un valore basso di Recall indica la presenza di intrusioni non rilevate.

$$Recall \ \ (TPR) = \frac{TP}{TP + FN}$$

- *True Negative Rate* (TNR): È chiamata anche *Specificity*, indica la capacità del modello di classificare correttamente il traffico benigno. È una metrica complementare alla Recall e aiuta a capire se il modello tende a sovrastimare la pericolosità della rete.

$$TNR = \frac{TN}{TN + FP}$$

- *F1-Score*: Consiste nella media armonica tra Precision e Recall, l'F1-Score è una metrica molto utilizzata, in quanto fornisce un punteggio unico che bilancia la necessità di non perdere attacchi (Recall) con quella di non generare troppi falsi allarmi (Precision).

$$F1\text{-}Score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

5.1.2 Area Under Time (AUT)

Mentre le metriche precedentemente descritte forniscono una valutazione puntuale in un preciso istante t, il paradigma del *Continual Learning* richiede di misurare come queste prestazioni si evolvono nel tempo. Per questo motivo, è stata introdotta anche la metrica *AUT (Area Under Time)* [16].

L'AUT viene definita come l'area sottesa alla curva delle prestazioni nel tempo. La formula per il calcolo è la segunete:

$$AUT(P, N) = \frac{1}{N-1} \sum_{k=1}^{N-1} \frac{P(X_{k+1}) + P(X_k)}{2}$$

Dove:

- $P(X_k)$ è il valore della metrica calcolato nello slot temporale k.

- N è il numero totale di slot temporali (nel nostro caso, 13 mesi).
- $\frac{1}{N-1}$ è il fattore di normalizzazione che garantisce che $AUT \in \{0, 1\}$

Un classificatore perfetto, totalmente immune al concept drift, otterrebbe un valore di $AUT = 1$. Questa metrica risulta particolarmente efficace per confrontare diverse soluzioni di *Continual Learning* e *Machine Unlearning*, in quanto fornisce un singolo valore scalare che riassume l'efficacia e la resilienza del sistema nell'intero scenario sperimentale.

5.2 Pre-Deployment

Lo scenario di pre-deployment rappresenta il punto iniziale della valutazione. In questa fase, il modello è stato addestrato e testato su dati prelevati dallo stesso intervallo temporale (da Settembre 2016 ad Aprile 2017), seguendo una suddivisione dei set di training e di testing rispettivamente del 70% e del 30%.

L'obiettivo di questa analisi è di verificare la validità delle feature estratte e la capacità dell'algoritmo di apprendere i patterne delle botnet presenti nel traffico iniziale.

Performance di base

In condizioni di stazionarietà, il modello mostra una separazione quasi netta tra le classi. Infatti, i risultati ottenuti nella Tabella 5.1 evidenziano come i valori di falsi positivi e falsi negativi siano estremamente ridotti.

Metrica	**RF**	**LR**	**MLP**
Precision	0.999	0.994	0.986
Recall (TPR)	0.999	0.979	0.993
F1-Score	0.999	0.991	0.990
TNR	0.999	0.998	0.986

Tabella 5.1: Performance nel mese di training (Pre-deployment).

Come si può notare dai dati nella Tabella 5.1, si può confermare che il classificatore è estremamente efficace nel riconoscere le minacce note, generando pochissimi falsi allarmi.

Considerazioni sull'AUT

In questo primo scenario statico (composto da un unico slot temporale), l'AUT non è ancora calcolabile, in quanto richiede una distribuzione temporale su più slot per l'applicazione della formula.

Il risultato delle metriche fungerà quindi da termine di paragone per gli scenari successivi, in cui ogni calo di tali metriche sarà attribuibile esclusivamente al *Concept Drift* nello scenario clean o a tentativi di *Adversarial Poisoning* nello scenario adversarial. Questa verifica ci permetterà di isolare e misurare con precisione le performance della strategia di unlearning proposta.

5.3 Risultati su scenario clean

In questo scenario, il sistema viene valutato su un arco temporale di 13 mesi (considerando anche le prestazioni iniziali di pre-deployment, da Aprile 2017 ad Aprile 2018). L'obiettivo è analizzare come il NIDS reagisce al naturale mutamento del traffico di rete (ossia il *Concept Drift*) e misurare la capacità delle strategie di aggiornamento di mantenere il modello performante senza interventi manuali.

5.3.1 Concept Drift

Il primo dato rilevante emerge dall'osservazione della strategia senza alcuna forma retraining (linea rossa nei grafici). Come si può notare in Figura 5.1, il modello addestrato unicamente sui dati iniziali subisce un degrado prestazionale drastico dopo pochi mesi di operatività.

Per esempio, la F1-Score, che nel pre-deployment era prossima a 1, crolla a valori inferiori a 0.10 in molti mesi. Questo fenomeno conferma che i pattern delle botnet evolvono rapidamente, rendendo un modello statico obsoleto in brevissimo tempo.

5.3.2 Continual Learning

L'introduzione del Continual Learning (linea blu nei grafici) permette di recuperare parte della capacità di rilevamento. Tuttavia, si nota come la sola strategia di Continual Learning presenta ancora molte instabilità. Infatti, in molti mesi, la Recall, dimostrando come l'aggiunta di nuovi dati

senza una pulizia accurata della conoscenza passata porta a fenomeni di catastrophic forgetting o di interferenza modello.

5.3.3 Machine Unlearning

La strategia proposta, che applica Continual Learning e Machine Unlearning (linea verde nei grafici), si dimostra ampiamente la più solida. Infatti, l'analisi sperimentale evidenzia come la curva relativa a questa metodologia non solo raggiunga picchi prestazionali più elevati, ma si distingua anche per una stabilità temporale significativamente maggiore.

Tale regolarità suggerisce che l'applicazione del Machine Unlearning agisca da meccanismo di regolarizzazione dinamica: Rimuovendo selettivamente i pattern obsoleti o le informazioni ridondanti, il framework evita il fenomeno di interferenza catastrofica tipico del Continual Learning. In sostanza, l'unlearning non si limita ad ottimizzare l'efficienza computazionale del sistema riducendo il carico di risorse necessarie, ma si rvela un elemento architetturale fondamentale per garantire l'affidabilità della classificazione nel lungo periodo, contrastando efficacemente il decadimento temporale indotto dal concept drift.

Infatti, osservando i grafici, la curva di questo approccio non è solo la più alta, ma è anche la più stabile. Il che significa che il Machine Unlearning risulta fondamentale sia per ridurre le risorse computazionali richieste dal sistema, sia per portare stabilità nella classificazione delle classi.

Per sintetizzare la capacità del framework di resistere al concept drift, è stata analizzata la metrica AUT applicata all'F1-Score, alla Precision e alla Recall.

Questa metrica risulta particolarmente significativa in quanto condensa l'intera serie storica delle prestazioni in un unico valore normalizzato, facilitando il confronto diretto tra le diverse strategie.

In particolare, i risultati evidenziano un divario netto tra le metodologie, dove si nota a maggior ragione come l'integrazione del Machine Unlearning sia in grado di mitigare efficacemente il concept drift, avvicinandosi alle prestazioni del modello in fase di pre-deployment.

Figura 5.1: Performance della F1 nello scenario clean

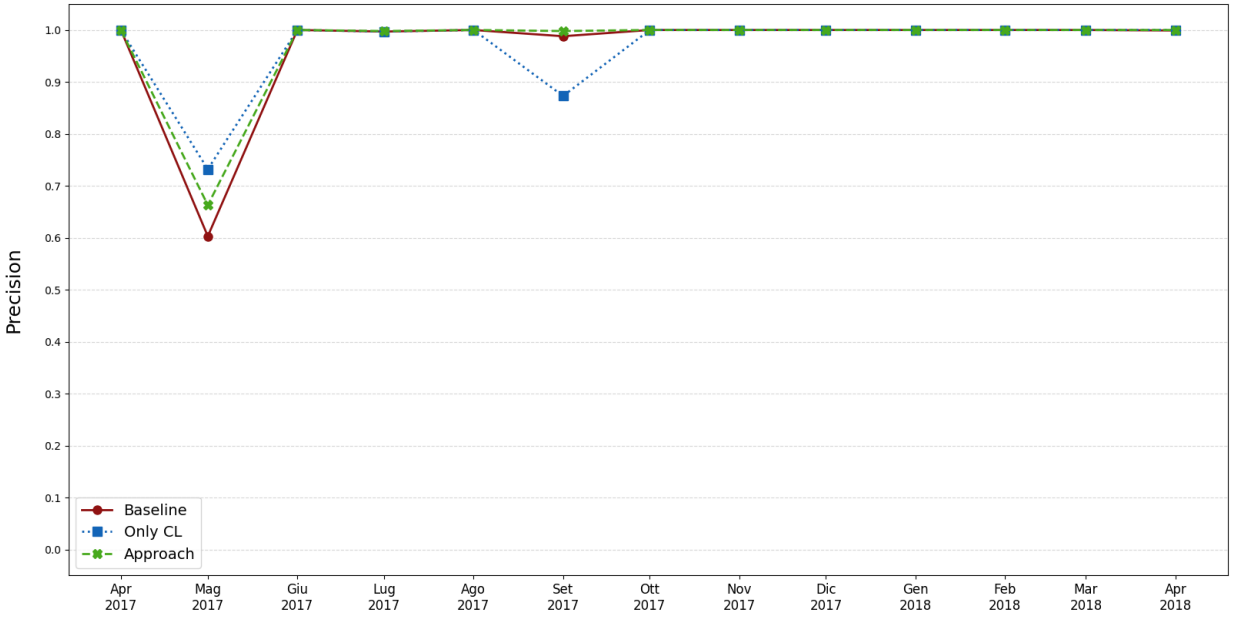

Figura 5.2: Performance della Precision nello scenario clean

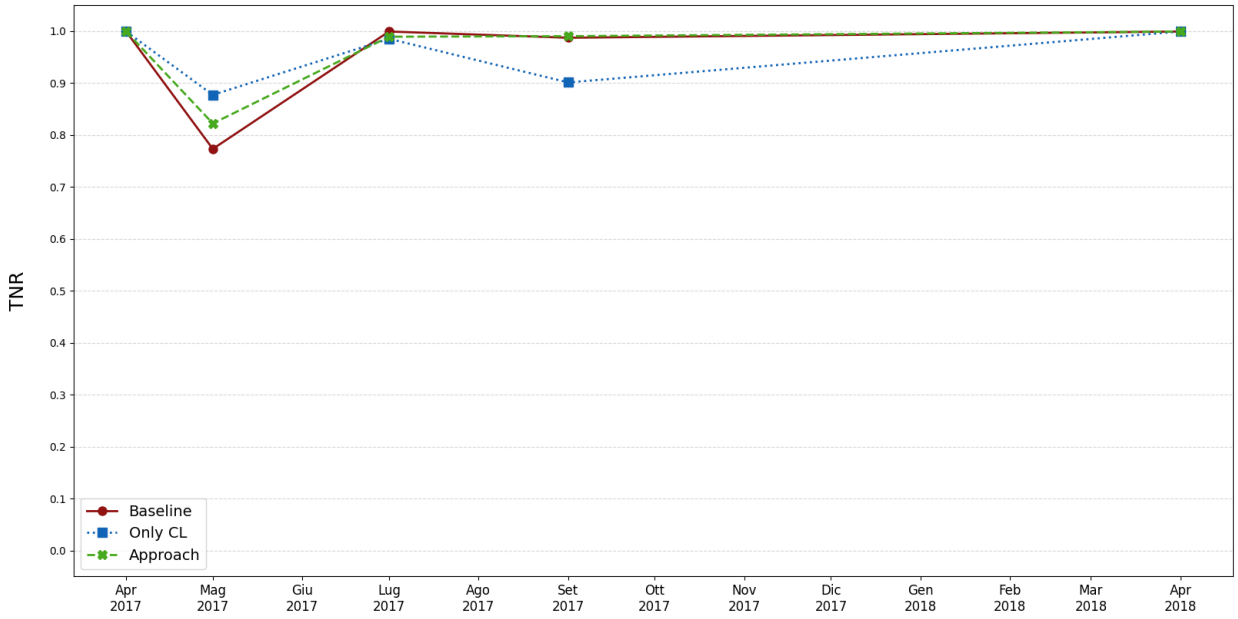

Figura 5.3: Performance della TNR (True Negative Rate) nello scenario clean

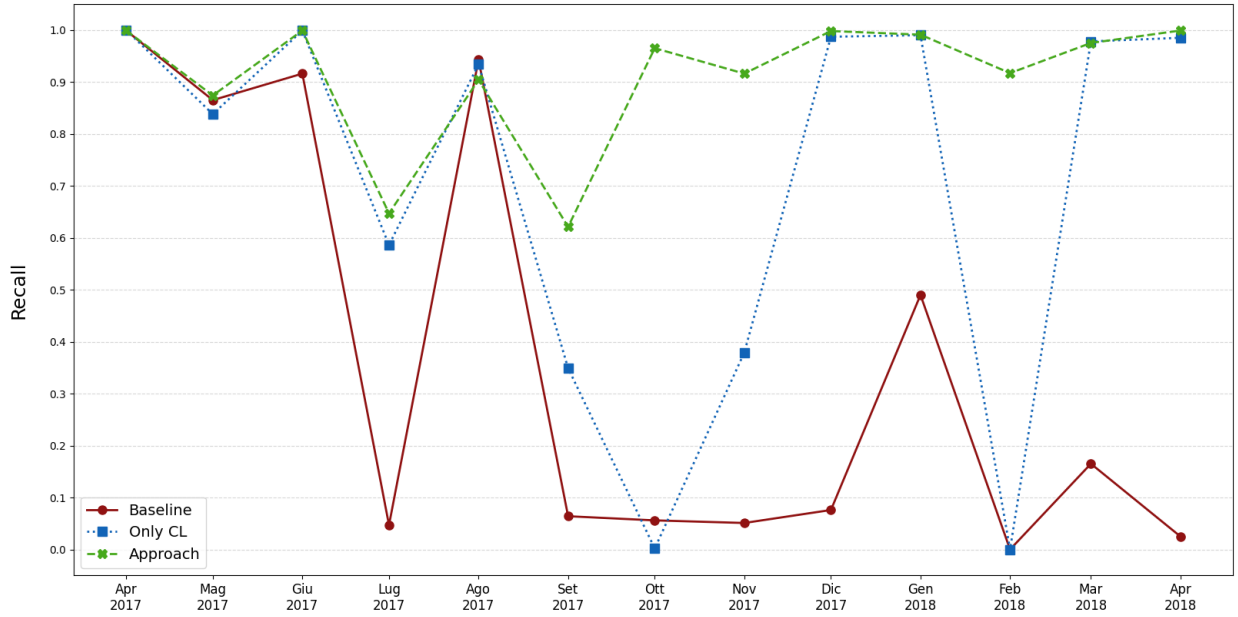

Figura 5.4: Performance della Recall nello scenario clean

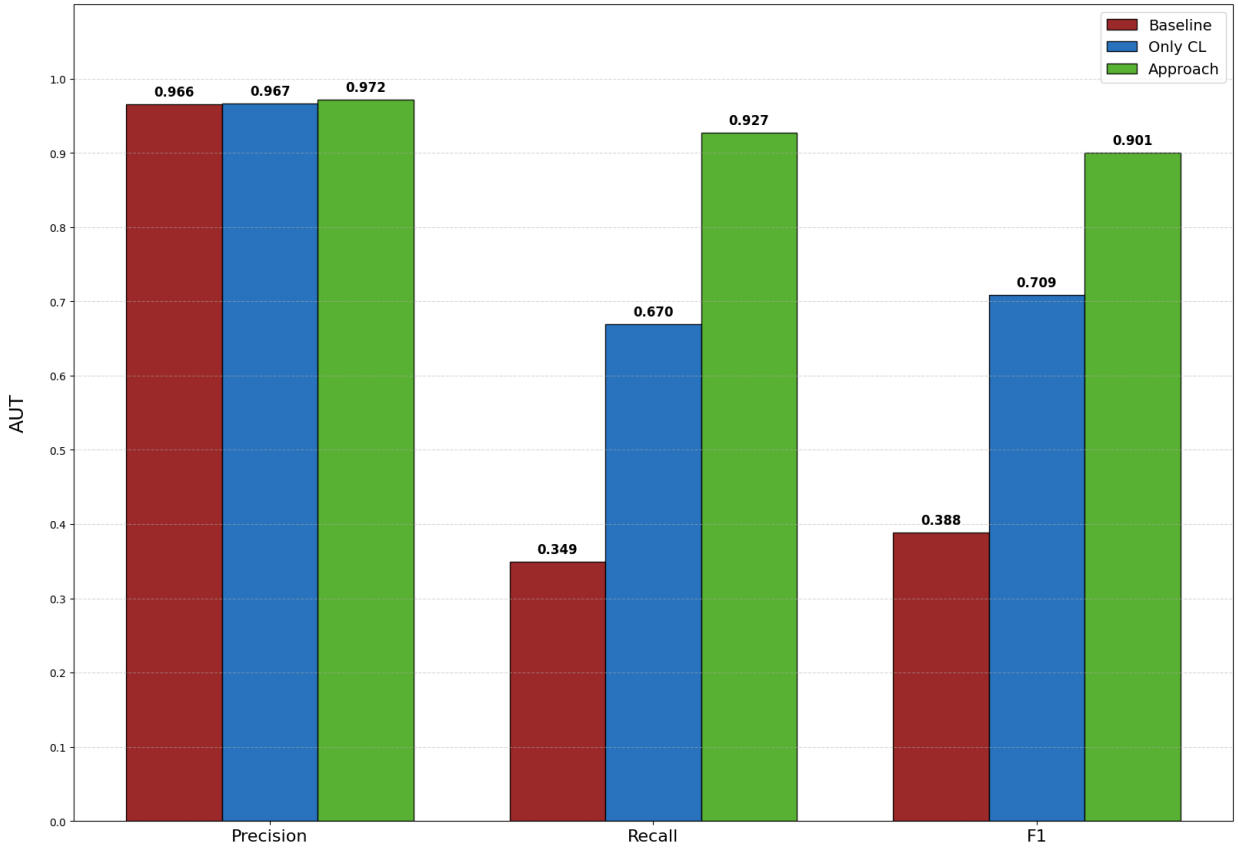

Figura 5.5: Performance dell'AUT nello scenario clean

5.3.4 Generalizzabilità del framework: Risultati su LR e MLP

Per validare la robustezza della strategia proposta e confermare che i benefici dell'utilizzo di Continual Learning e Machine Unlearning non siano limitati ad una singola architettura di classificazione, il framework è stato testato utilizzando anche altri due modelli di natura intrinsecamente differente: La *Logistic Regression* (LR), come rappresentante di modelli lineari, e il *Multi-Layer Perceptron*, per valutare l'efficacia anche su reti neurali.

Classificatore	Strategia	AUT F1	AUT Precision	AUT Recall
Random Forest	Baseline	0.338	0.966	0.349
	Only CL	0.709	0.967	0.670
	Our approach	**0.901**	**0.972**	**0.927**
Logistic Regression	Baseline	0.499	0.974	0.446
	Only CL	0.707	0.982	0.648
	Our approach	**0.889**	**0.982**	**0.832**
MLP (Rete Neurale)	Baseline	0.545	0.949	0.490
	Only CL	0.867	0.978	0.825
	Our approach	**0.940**	**0.979**	**0.910**

Tabella 5.2: Confronto delle prestazioni medie (AUT) tra diverse architetture e strategie di difesa dal concept drift.

L'analisi comparativa dei risultati, sintetizzata nella Tabella 5.2, evidenzia come l'integrazione di Continual Learning e Machine Unlearning (indicata come "*Our approach*") sovraperformi costantemente sia la configurazione *baseline* che l'approccio con solo Continual Learning ("*Only CL*"), indipendentemente dal classificatore sottostante.

Questo dato conferma che il framework è intrinsecamente indipendente dal modello adottato, rendendolo una soluzione versatile per diverse infrastrutture di rete.

5.3.5 Analisi dei risultati

Gli esami nei vari grafici confermano che l'approccio proposto garantisce un incremento prestazionale sistematico rispetto alla baseline per tutte le architetture testate. In particolare, si possono notare una serie di caratteristiche:

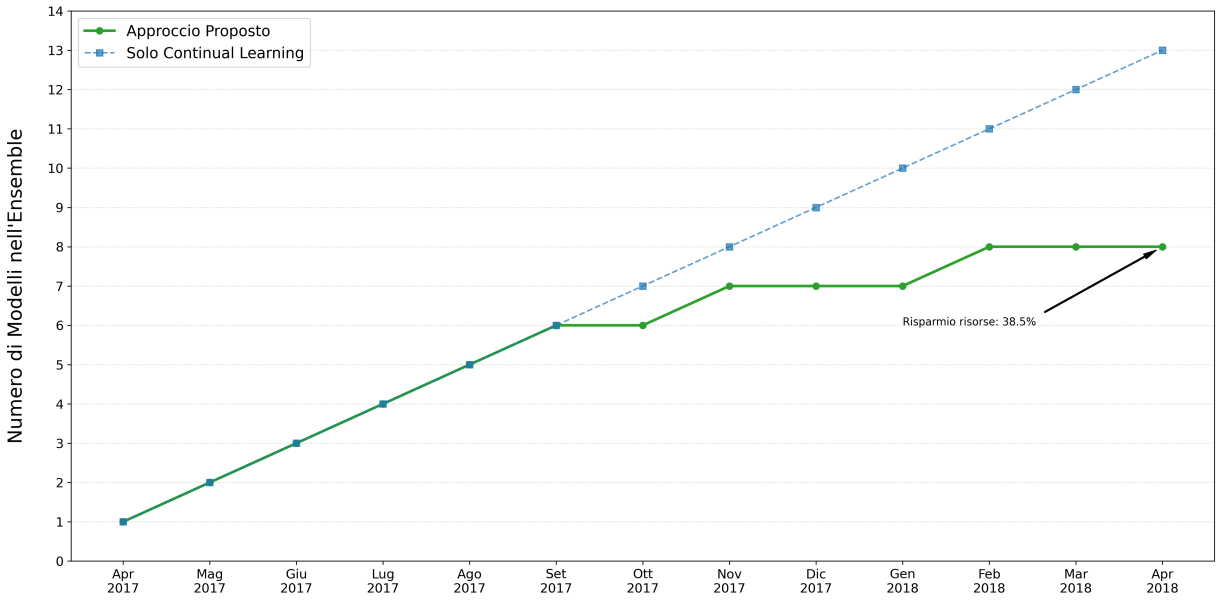

Figura 5.6: Confronto dell'evoluzione temporale del numero di modelli nell'ensemble. La linea dell'approccio proposto mostra l'effetto del Data Pruning nel mantenere la dimensionalità ridotta rispetto alla crescita lineare del solo Continual Learning.

- *Recall elevata*: In tutti i classificatori, il passaggio dalla configurazione baseline a quella proposta comporta un recupero della recall superiore al 50%, dimostrando che il framework mitiga efficacemente il decadimento indotto dal concept drift.

- *Stabilità*: Le performance mese per mese sono nettamente più stabili rispetto agli altri approcci, suggerendo che l'utilizzo del buffer contenente le feature più importanti e la rimozione eventuale di un modello dall'ensemble siano sufficienti a mantenere la stabilità del modello nel tempo.

- *Indipendenza del framework*: Il fatto che Random Forest, Logistic Regression e Multi-Layer Perceptron beneficino in modo analogo della strategia di Machine Unlearning, prova che la rimozione dei modelli obsoleti agisce come regolarizzatore universale, preservando le performance del sistema senza richiedere uno specifico classificatore di base.

Un ulteriore e significativo vantaggio derivante dall'integrazione del Machine Unlearning riguarda l'efficienza computazionale. Come evidenziato nel grafico di Figura 5.6, mentre un approccio basato esclusivamente sul Continual Learning incrementale porterebbe a un'espansione lineare e indefinita della Modular Network (fino a 18 modelli), l'attivazione del Data Pruning permette di

mantenere la dimensionalità dell'ensemble significativamente inferiore. Questa riduzione proattiva si traduce in una minore occupazione di risorse, fattore critico per un'applicazione in contesti di monitoraggio di rete a lungo termine.

Al fine di garantire una migliore chiarezza espositiva ed evitare una ridondanza di informazioni, l'analisi puntuale dell'andamento mensile delle performance per LR e MLP non è riportata graficamente in questa sede. Tuttavia, l'evoluzione temporale di tali modelli riflette le dinamiche di adattamento già osservate per il Random Forest, con un recupero della stabilità coincidente con l'attivazione dei cicli di unlearning. In ogni caso, l'analisi si affida comunque alla metrica *AUT*, che condensa efficacemente l'andamento temporale. I grafici di dettaglio e le serie storiche complete sono consultabili nella repository GitHub[1] del progetto per scopi di riproducibilità sperimentale.

5.4 Risultati su scenario adversarial

In quest'ultimo scenario viene analizzata la capacità del framework di rilevare minacce manipolate per eludere i sistemi di retraining. Infatti, mentre il *Concept Drift* rappresenta un'evoluzione passiva delle distribuzioni dei dati, l'attacco *Adversarial* è una strategia attiva in cui l'attaccante introduce perturbazioni nel traffico in input per alterare la rappresentazione dello spazio delle feature vista dal modello.

Per riassumere l'attacco, è stato simulato un attacco realistico con una limitata conoscenza del sistema. In particolare, è stata implementata una tecnica di *poisoning* dei dati senza alcuna conoscenza sul modello.

Sono state manipolate tre feature: *TotPkts*, *SrcBytes* e *Dur*, incrementandone il valore e, per garantire la validità del flusso, modificando di conseguenza tutte le feature derivate. I valori adottati per le modifiche sono visibili nella Tabella 4.7

Le perturbazioni sono state iniettate sia singolarmente che insieme in tre sotto-scenari, iniettandole nei primi 3,4 e 5 mesi di test per valutare la capacità di adattamento del framework.

L'analisi comparativa tra strategia di solo addestramento continuo (*Only CL*) e il framework proposto (*Our*) evidenzia una superiorità quasi sistematica di quest'ultimo. Infatti, le rare occorrenze in cui l'utilizzo del Machine Unlearning ha causato un'AUT della *Recall* inferiore al semplice

[1] `https://github.com/montaooo/Master-internship`

Scenario	Valore	Durata: 3 mesi		Durata: 4 mesi		Durata: 5 mesi	
		Only CL	*Our*	*Only CL*	*Our*	*Only CL*	*Our*
SrcBytes	1	0.716	**0.836**	0.679	**0.848**	0.692	**0.885**
	64	0.634	**0.761**	0.651	**0.786**	0.651	**0.883**
	1024	0.349	**0.352**	0.461	**0.379**	0.473	**0.585**
TotPkts	1	0.605	**0.710**	0.599	**0.858**	0.574	**0.824**
	15	0.357	**0.379**	0.430	**0.380**	0.488	**0.575**
	100	0.331	**0.426**	0.494	**0.580**	0.501	**0.564**
Duration	1	0.374	**0.614**	0.467	**0.569**	0.504	**0.621**
	15	0.375	**0.620**	0.437	**0.497**	0.521	**0.548**
	120	0.352	**0.635**	0.485	**0.480**	0.489	**0.544**
All Features	(1, 1, 1)	0.364	**0.446**	0.430	**0.498**	0.486	**0.588**
	(64, 15, 15)	0.330	**0.558**	0.428	**0.509**	0.482	**530**
	(1024, 100, 120)	0.354	**0.492**	0.475	**0.427**	0.480	**0.518**

Tabella 5.3: Confronto delle prestazioni (AUT Recall) tra scenario con solo CL e con il framework proposto al variare della durata e dell'intensità dell'attacco.

addestramento incrementale si verificano esclusivamente in scenari con iniezioni elevate, e quindi più complesse da gestire.

Al di fuori di queste eccezioni, il framework dimostra anche una capacità difensiva maggiore. Mentre il Continual Learning standard accumula in modo indiscriminato nuovi modelli, rischiando di "inquinare" l'ensemble con pattern adversarial, il Machine Unlearning riesce anche ad agire come un filtro che epura costantemente la conoscenza obsoleta e dannosa, rendendolo migliore sia nel breve che nel lungo termine anche sotto questo aspetto.

In conclusione, questa fase di valutazione ha confermato che la strategia proposta risolve efficacemente il problema del concept drift e, contemporaneamente, migliora anche la robustezza del NIDS contro attacchi adversarial. Il Machine Unlearning non si dimostra quindi solo come un'ottimizzazione per le risorse computazionali ma anche come un elemento architetturale fondamentale per garantire affidabilità e sicurezza dei sistemi di rilevamento nel lungo periodo.

6. Conclusione

In questa tesi è stato presentato e validato un framework originale per la protezione delle reti aziendali, volto a superare i limiti strutturali dei sistemi di Intrusion Detection statici. Il cuore della ricerca ha riguardato la gestione dinamica della conoscenza in presenza di concept drift, un fenomeno ineludibile che compromette l'efficacia dei modelli di Machine Learning nel lungo periodo.

L'approccio proposto ha integrato sinergicamente strategie di Continual Learning, per l'acquisizione incrementale di nuove minacce tramite una *Modular Network* ed *Experience Replay*, e tecniche di Machine Unlearning, utilizzate come strumento di manutenzione proattiva per la rimozione di informazioni obsolete o degradate attraverso il *Data Pruning*. La traduzione tecnica di questa metodologia ha permesso di creare un sistema capace di evolvere autonomamente, mantenendo un equilibrio costante tra la stabilità della memoria storica e la plasticità necessaria per affrontare nuovi scenari d'attacco.

6.1 Discussione dei risultati

La fase sperimentale, condotta sul dataset MCFP per un arco temporale di dodici mesi, ha confermato la validità del framework attraverso diverse evidenze:

- La configurazione priva di aggiornamenti ha confermato l'estrema vulnerabilità dei modelli statici. Già dopo i primi mesi di operatività, il sistema ha subito un degrado prestazionale drastico, con la metrica F1-Score che è crollata da valori prossimi all'unità a punteggi inferiori a 0.10. Questo scenario dimostra che il concept drift naturale e l'evoluzione delle botnet rendono un NIDS tradizionale rapidamente obsoleto e incapace di garantire la sicurezza perimetrale.

- L'integrazione del solo Continual Learning ha permesso di recuperare parte della capacità di rilevamento, ma ha evidenziato forti limitazioni in termini di stabilità. Le performance hanno mostrato ampie oscillazioni mensili, dovute a fenomeni di interferenza tra i modelli e

all'accumulo di conoscenza obsoleta. Sebbene questa strategia resista parzialmente al drift, la mancanza di una pulizia dei dati impedisce il mantenimento di un decision boundary costante.

- La configurazione completa, che combina Continual Learning e Machine Unlearning, si è dimostrata la soluzione più solida e performante. L'applicazione del *Data Pruning* ha agito come un meccanismo di regolarizzazione dinamica, eliminando il rumore dei modelli obsoleti o inquinati. I grafici evidenziano una curva di performance non solo più elevata, ma estremamente stabile nel tempo, avvicinandosi ai livelli di accuratezza registrati nella fase di pre-deployment.

  Tale configurazione si è rivelata utile anche in situazione di attacchi di tipo adversarial. Essa ha permesso di identificare e dimenticare i moduli inquinati da tentativi di poisoning delle feature, garantendo una difesa superiore rispetto ai classici metodi di addestramento continuo.

- I test eseguiti su Random Forest, Logistic Regression e Multi-Layer Perceptron hanno provato che i benefici della strategia sono indipendenti dal classificatore scelto, rendendo il framework una soluzione versatile e scalabile per infrastrutture di rete reali.

In sintesi, mentre il solo Continual Learning fornisce la plasticità necessaria per apprendere, è il Machine Unlearning a conferire al sistema la stabilità indispensabile per un impiego in contesti di produzione, garantendo che le decisioni dell'ensemble siano sempre guidate dalla conoscenza più attuale e affidabile.

6.2 Sviluppi futuri

Nonostante i risultati positivi ottenuti, il lavoro delinea diverse direzioni per ricerche future volte a perfezionare la resilienza e l'efficienza del framework:

- *Ottimizzazione della robustezza adversarial*: Sebbene l'approccio proposto si sia dimostrato superiore al solo apprendimento continuo nel rilevamento di minacce manipolate, l'analisi ha evidenziato che il sistema non è ancora ottimizzato per garantire un'affidabilità assoluta in presenza di attacchi adversarial. Ricerche future potrebbero indagare l'integrazione di tecniche di Adversarial Training specifiche per i NIDS, al fine di rendere il buffer di memoria intrinsecamente più resistente ai tentativi di iniezione di dati malevoli perturbati.

- *Ottimizzazione dinamica del Machine Unlearning*: Un'evoluzione significativa del framework riguarda il superamento dei vincoli statici attualmente imposti alla fase di Data Pruning. Nel sistema implementato, la rimozione dei moduli è vincolata a una dimensione minima dell'ensemble e alla cancellazione di un singolo modello per ogni ciclo mensile. Un possibile sviluppo futuro può prevedere l'impiego del *Reinforcement Learning* per automatizzare la selezione e il numero di modelli da rimuovere. Un agente intelligente potrebbe apprendere, attraverso una funzione di ricompensa basata sul bilanciamento tra accuratezza e carico computazionale, se sia più vantaggioso eliminare molteplici moduli obsoleti in un unico step o conservare una memoria storica più estesa in periodi di stazionarietà del traffico .

In conclusione, questa tesi dimostra che il superamento della staticità dei modelli ML-NIDS non risiede solo nella capacità di apprendere costantemente, ma soprattutto nella capacità di dimenticare selettivamente. L'integrazione di Continual Learning e Machine Unlearning si conferma dunque un paradigma essenziale per la costruzione di sistemi di difesa perimetrale realmente resilienti e sostenibili nel tempo.

Ringraziamenti

Desidero ringraziare i Professori Mirco Marchetti e Mauro Andreolini; il loro supporto e i loro consigli sono stati fondamentali nel guidarmi verso la scelta di questo percorso di tirocinio e tesi, permettendomi di lavorare su un progetto stimolante e in linea con i miei studi.

Vorrei inoltre ringraziare l'Ing. Dimitri Galli, che mi ha seguito quotidianamente durante l'attività in laboratorio. Lo ringrazio per avermi trasmesso le competenze necessarie e per avermi accompagnato in questo percorso, risolvendo con pazienza ogni impervisto sperimentale.

Ringrazio inoltre tutti i colleghi del laboratorio SECloud per l'accoglienza e per aver reso il clima di lavoro sereno e stimolante.

Un ringraziamento speciale va alla mia famiglia. Ai miei genitori, per il sostegno costante e per non avermi mai fatto mancare nulla; e a mio fratello Matteo, per esserci stato in ogni momento di dubbio e per il suo prezioso incoraggiamento.

Ringrazio sinceramente anche Edoardo, Andrea e Francesco, compagni di viaggio in questo percorso magistrale. Vi ringrazio per i confronti, le sessioni di studio condivise e per aver reso questi due anni decisamente più leggeri e divertenti.

Un ulteriore ringraziamento va a Francesco, Daniele, Alberto e Paolo; per essere stati al mio fianco non solo in palestra tra allenamenti e partite, ma anche in tutte le serate e i momenti di svago trascorsi insieme.

Infine, il ringraziamento più importante va alla mia morosa Giorgia. Grazie per essere stata al mio fianco in ogni momento, per aver creduto in me anche quando smettevo di farlo io stesso e per avermi dato la carica nei momenti di difficoltà. Senza il tuo supporto e la tua pazienza, questo traguardo non sarebbe stato lo stesso.

Bibliografia

[1] Zeeshan Ahmad et al. «Network intrusion detection system: A systematic study of machine learning and deep learning approaches». In: *Transactions on Emerging Telecommunications Technologies* 32.1 (2021), e4150. DOI: https://doi.org/10.1002/ett.4150. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/ett.4150. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/ett.4150.

[2] Evidently AI. *What is concept drift?* URL: https://www.evidentlyai.com/ml-in-production/concept-drift#what-is-concept-drift.

[3] Giuseppina Andresini et al. «INSOMNIA: Towards Concept-Drift Robustness in Network Intrusion Detection». In: *Proceedings of the 14th ACM Workshop on Artificial Intelligence and Security*. AISec '21. Virtual Event, Republic of Korea: Association for Computing Machinery, 2021, pp. 111–122. ISBN: 9781450386579. DOI: 10.1145/3474369.3486864. URL: https://doi.org/10.1145/3474369.3486864.

[4] Giovanni Apruzzese, Aurore Fass e Fabio Pierazzi. «When Adversarial Perturbations meet Concept Drift: An Exploratory Analysis on ML-NIDS». In: *Proceedings of the 2024 Workshop on Artificial Intelligence and Security*. AISec '24. Salt Lake City, UT, USA: Association for Computing Machinery, 2024, pp. 149–160. ISBN: 9798400712289. DOI: 10.1145/3689932.3694757. URL: https://doi.org/10.1145/3689932.3694757.

[5] Giovanni Apruzzese et al. «The Role of Machine Learning in Cybersecurity». In: *Digital Threats* 4.1 (mar. 2023). DOI: 10.1145/3545574. URL: https://doi.org/10.1145/3545574.

[6] Lucas Bourtoule et al. «Machine Unlearning». In: *2021 IEEE Symposium on Security and Privacy (SP)*. 2021, pp. 141–159. DOI: 10.1109/SP40001.2021.00019.

[7] Anna L. Buczak e Erhan Guven. «A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection». In: *IEEE Communications Surveys and Tutorials* 18.2 (2016), pp. 1153–1176. DOI: 10.1109/COMST.2015.2494502.

[8] Hervé Debar, Marc Dacier e Andreas Wespi. «A revised taxonomy for intrusion-detection systems». In: *Annals of Telecommunications* 55 (2000).

[9] Sajad Einy, Cemil Oz e Yahya Dorostkar Navaei. «The Anomaly- and Signature-Based IDS for Network Security Using Hybrid Inference Systems». In: *Mathematical Problems in Engineering* 2021.1 (2021), p. 6639714. DOI: https://doi.org/10.1155/2021/6639714. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1155/2021/6639714. URL: https://onlinelibrary.wiley.com/doi/abs/10.1155/2021/6639714.

[10] Moses Garuba, Chunmei Liu e Duane Fraites. «Intrusion techniques: Comparative study of network intrusion detection systems». In: *Fifth international conference on information technology: new generations (itng 2008)*. IEEE. 2008, pp. 592–598.

[11] Gigaspaces. *Real-time Data Integration vs. Batch Data Integration*. URL: https://www.gigaspaces.com/blog/real-time-data-integration-vs-batch-data-integration.

[12] Roberto Jordaney et al. «Transcend: Detecting concept drift in malware classification models». In: *26th USENIX security symposium (USENIX security 17)*. 2017, pp. 625–642.

[13] Korbinian Koch e Marcus Soll. «No Matter How You Slice It: Machine Unlearning with SISA Comes at the Expense of Minority Classes». In: *2023 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. 2023, pp. 622–637. DOI: 10.1109/SaTML54575.2023.00047.

[14] Microsoft. *Che cos'è un centro operazioni per la sicurezza (SOC)?* URL: https://www.microsoft.com/it-it/security/business/security-101/what-is-a-security-operations-center-soc.

[15] Microsoft. *Introduzione a SIEM*. URL: https://www.microsoft.com/it-it/security/business/security-101/what-is-siem.

[16] Feargus Pendlebury et al. «TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time». In: *28th USENIX Security Symposium*. USENIX Sec. Santa Clara, CA: USENIX Association, 2019.

[17] S2Lab. *Tesseract ML release*. URL: https://github.com/s2labres/tesseract-ml-release.

[18] Hami Satilmiş, Sedat Akleylek e Zaliha Yüce Tok. «A Systematic Literature Review on Host-Based Intrusion Detection Systems». In: *IEEE Access* 12 (2024), pp. 27237–27266. DOI: 10.1109/ACCESS.2024.3367004.

[19] Iman Sharafaldin, Arash Habibi Lashkari, Ali A Ghorbani et al. «Toward generating a new intrusion detection dataset and intrusion traffic characterization.» In: *ICISSp* 1.2018 (2018), pp. 108–116.

[20] Stratosphere. *Stratosphere Laboratory Datasets*. Retrieved March 13, 2020, from https://www.stratosphereips.org/datasets-overview. 2015.

[21] Ultralytics. *Apprendimento continuo*. URL: https://www.ultralytics.com/it/glossary/continual-learning.

[22] Liyuan Wang et al. «A Comprehensive Survey of Continual Learning: Theory, Method and Application». In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46.8 (2024), pp. 5362–5383. DOI: 10.1109/TPAMI.2024.3367329.

[23] Wikipedia. *Botnet*. 2026. URL: https://it.wikipedia.org/wiki/Botnet.

[24] Wikipedia. *Independent and identically distributed random variables*. 2025. URL: https://en.wikipedia.org/wiki/Independent_and_identically_distributed_random_variables.

[25] Heng Xu et al. «Machine Unlearning: A Survey». In: *ACM Comput. Surv.* 56.1 (ago. 2023). ISSN: 0360-0300. DOI: 10.1145/3603620. URL: https://doi.org/10.1145/3603620.

[26] J. Zhang e M. Zulkernine. «A hybrid network intrusion detection technique using random forests». In: *First International Conference on Availability, Reliability and Security (ARES'06)*. 2006, 8 pp.–269. DOI: 10.1109/ARES.2006.7.